INITIATION A LA PROGRAMMATION JAVA

Université Cheikh Anta DIOP



Département de Mathématiques informatique
Section informatique

HISTORIQUE

- Initialement, projet de la société Sun (rachetée depuis par Oracle) pour l'électronique grand public (1991)
  • Transformé en langage pour le Web, sous le nom de « Java », grâce à sa portabilité (1994/95)
  • Lancement officiel en mai 1995
  • Après l'engouement pour les applets, Java est progressivement reconnu comme un langage à part entière
  • Langage de programmation sans doute le plus utilisé aujourd'hui :
- – plusieurs millions de développeurs Java
- – nombreux outils de développement

INTERETS DE JAVA

- logiciels portables
- programmes fiables (rigueur du langage => peu de bugs)
- développement rapide
- pages Web interactives (via les « applets »)
- logiciels (ou briques logicielles) téléchargeables, éventuellement automatiquement.
- gestion de la sécurité (par défaut, accès restreint aux ressources locales pour les applets)

CARACTERISTIQUES DE JAVA

- **Un langage orienté-objet :**
  - portable
  - compilé puis interprété (bytecode+JVM)
  - robuste (typage fort, garbage collector)
- **Une énorme librairie de classes standards**

JRE–JDK–ECLIPSE

- Plusieurs implantations disponibles du langage Java, dont celles d'origine
- ORACLE/SUN (créateur du langage)
- • « **JRE » (Java Runtime Environment)**
- • « **Java SE » (Standard Edition) « JDK » (Java Development Kit)**
- (gratuit) contient :
  - compilateur (javac)
  - interpréteur / machine virtuelle (java)
  - toute la librairie de classes standards
  - divers outils : génération doc (javadoc), visualisation d'applet
- • « **Eclipse » (gratuite)**
- • Les sources de Java sont disponibles

PRINCIPES de la POO

- Unité logique : l'objet
  - Objet est défini par
  - un état
  - un comportement
  - une identité

Voiture
➢Couleur = Bleue ➢Vitesse= 100

- État : représenté par des attributs (variables) qui stockent des valeurs
- Comportement : défini par des méthodes (procédures) qui modifient des états
- Identité : permet de distinguer un objet d'un autre objet

PRINCIPES de la POO

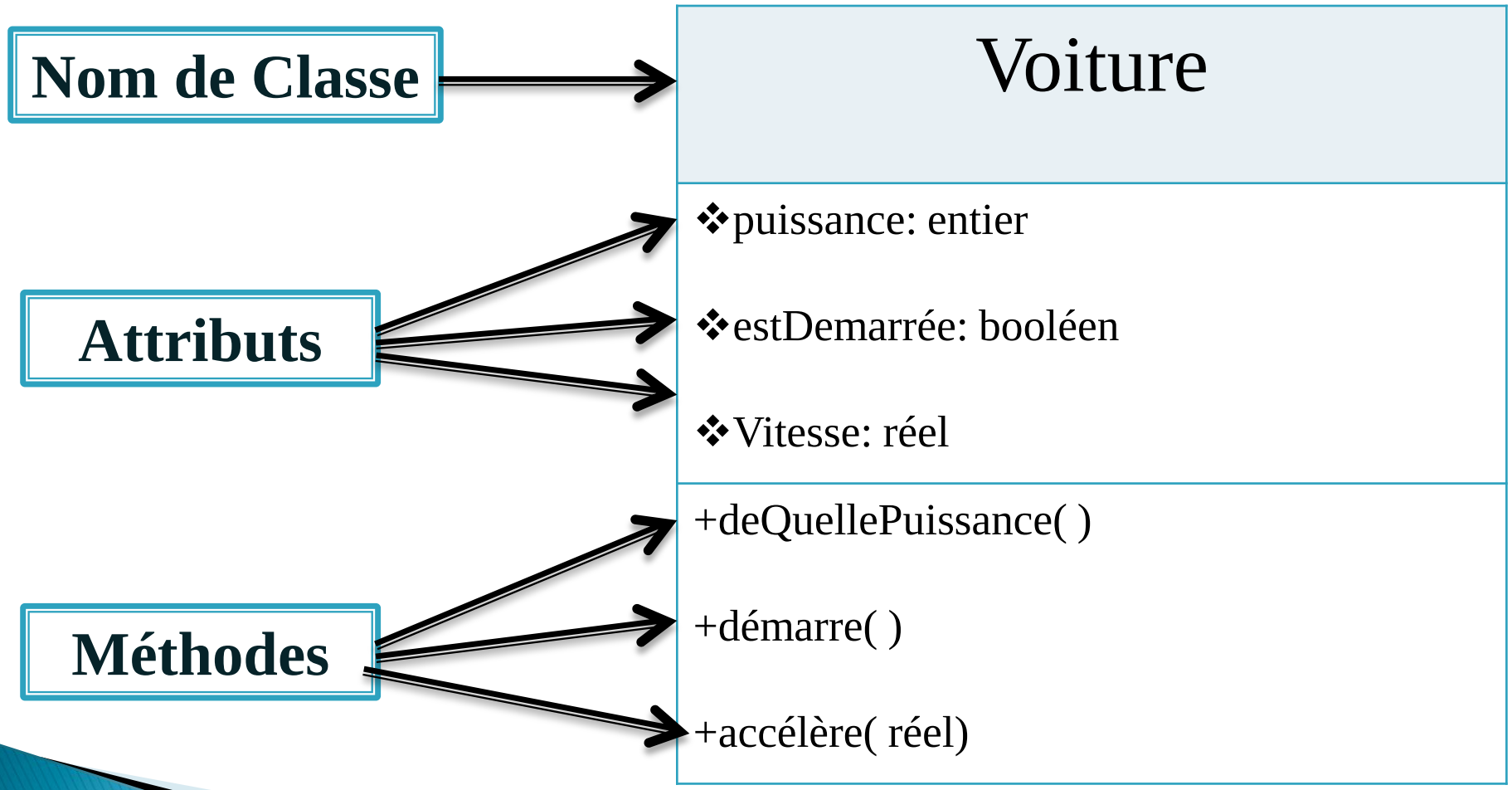
- Les objets communiquent entre eux par des messages
- Un objet peut recevoir un message qui déclenche une méthode qui modifie son état et / ou une méthode qui envoie un message à un autre objet



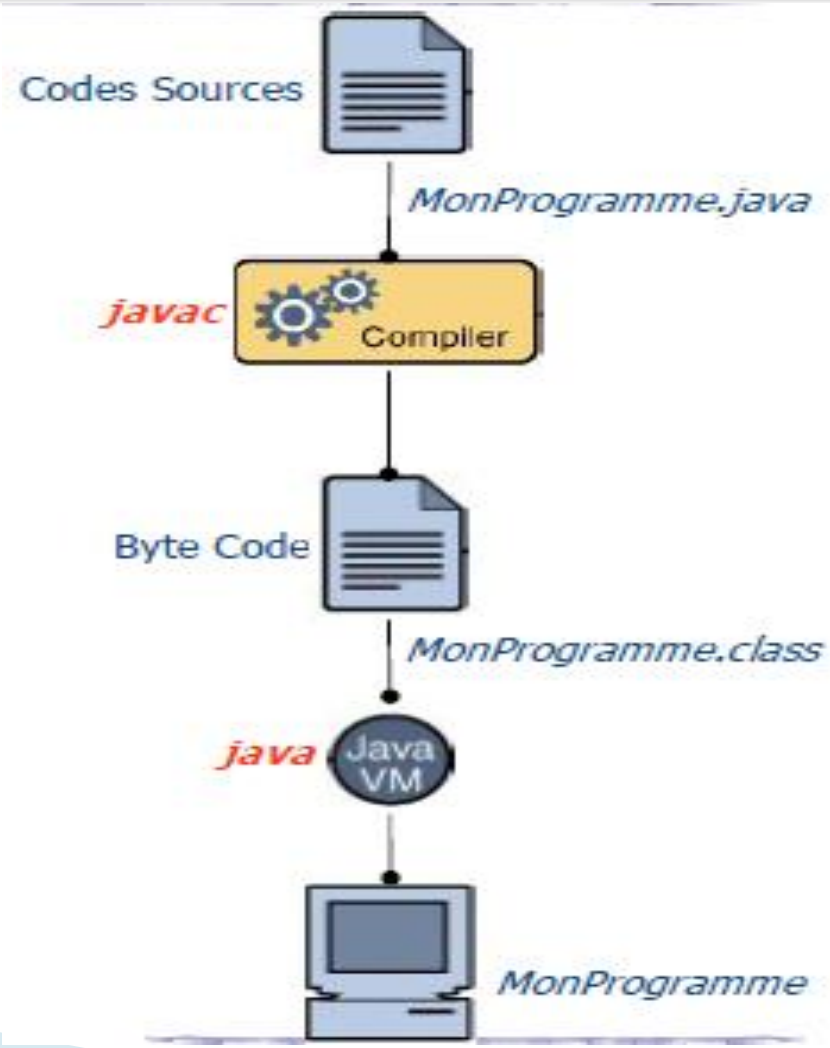
POO: Notion de Classe

- Les objets qui ont les mêmes états et les mêmes comportements sont regroupés : c'est une **classe.**
- Les classes servent de « moules » pour la création des objets.
- Un objet est une **instance d'une classe**
- Un programme OO est constitué de classes qui permettent de créer des objets qui s'envoient des messages.

POO: Notion de Classe



Principales étapes d'un développement



Premier exemple du programme JAVA

```
public class PremierProg
{
    public static void main (String[] argv)
    {
        System.out.println("Ola, mon
    Premier Programme");
  }
}
```

- public class PremierProg
  - Nom de la classe

Premier exemple du programme JAVA

- public static void main
  - La fonction principale équivalent à la fonction main du C/C++
- String[] argv
  - Permet de récupérer des arguments transmis au programme au moment de son lancement
- System.out.println("Ola, mon premier programme")
  - Méthode d'affichage dans la fenêtre console

MISE EN OEUVRE

- Pas de séparation entre définition et codage des opérations.
  - Un seul fichier « NomDeClasse.java »

- **Compilation**

Nom de la classe = Nom du fichier java

  - javac NomDeClasse.java
  - Génération d'un fichier Byte-Code « NomDeClasse.class »

MISE EN OEUVRE

- Exécution

Ne pas mettre l'extension « .class » lors de l'exécution

  - java NomDeClasse
  - Choisir la classe principale à exécuter

VARIABLES ET TYPES

- **notion de variable**
  - nom (identificateur) + type+zone mémoire
- **en Java, deux grandes catégories de types**
- types « **primitifs** » **(entiers, flottants, …)**
- **références à des types « composites » :**
  - Tableaux
  - énumérations
  - objets
  - interfaces

TYPES PRIMITIFS

- boolean
- char: 16-bit
- byte: entier 8-bit
- short: entier 16-bit
- int: entier 32-bit
- long: entier (signé) 64-bit
- float: flottant32-bit
- double: flottant 64-bit

BOOLEENS : boolean

- 2 valeurs : true ou false
- type retourné par les opérateurs de comparaison
- type attendu dans tous les tests

ENTIERS

- valeurs min/max :
- byte = [-128; +127]
- short = [-32768 ; +32767]
- int = [-2.147.483.648 ; +2.147.483.647]
- long = [-9,223... 1018 ; +9,223... 1018]
- • conversion automatique seulement vers les types entiers plus grands.

Exemple: (int->long) et vers les types flottants

CARACTERES char

- 16-bit 65536 valeurs : quasiment tous les caractères de toutes les écritures !
- ne sont affichables que si le système possède les polices de caractères adéquates !
- littéraux entre simples quotes : 'a' 'Z‘
- caractères spéciaux : '\n' '\t' '\b' '\\‘
- convertible automatiquement en int ou long (et manuellement en byte ou short)
- inversement, (char)val est le caractère dont le code Unicode est l'entier val

FLOTTANTS

- float x = 2.5; // Erreur
- double y = 2.5; // OK
- valeurs f ou F pour spécifier un float :
- float x = 2.5f;
- conversion automatique:
  - seulement float -> double.
- la conversion « manuelle » en entier tronque la partie décimale
- float x = -2.8f;
- int i = (int) x; // => i==-2

CONSTANTES

- variable dont la valeur ne peut plus être changée une fois fixée
- se déclare avec le mot-clé final :
- final double PI = 3.14159;
- ce qui interdit d'écrire ensuite
- PI = 3.14; // ERREUR
- possibilité de calculer la valeur de la constante plus tard à l'exécution.
- // OK : constante « blanche
- // ...
- MAX_VAL =10

Déclaration et portée des variables

- déclaration préalable obligatoire
  - identificateurs :
  - caractères Unicode
  - commencent par une lettre, _ ou $
  - ensuite : lettre, chiffre, _ ou $
- exemples de déclaration:
  - int i;
  - float x, y, z;
  - char c = 'A', d;
  - boolean flag = true;
- initialisation obligatoire avant usage :
  - int i, j;

  j = i; // ERREUR : i non initialisé
- « portée » des variables (= zone de validité de la déclaration): jusqu'à la fin du *bloc englobant (voir plus loin)*ion préalable obligatoire

COMMENTAIRES

- Sur une ligne

```
//Commentaire -> fin de la ligne
int x; // Ligne partielle
```

- Sur plusieurs lignes

```
/* Commentaire qui s'étend sur plusieurs lignes */
```

- De documentation

```
/** Commentaire pour javadoc */
```

OPERATEURS

- affectation : =
- arithmétiques : +, -, *, /, %
- comparaisons : <, <=, >, >=, ==, !=
- booléens : &&, ||, &, |
- pré/post-incrémentation : ++
- pré/post-décrémentation : --
- création tableau ou objet (allocation mémoire) : new

Operateurs arithmétiques

Operateur	Fonction	usage
+	Addition	expr1+expr2
-	Soustraction	expr1_expr2
-	Changement de signe	-expr
*	Multiplication	expr1*expr2
/	Division	expr1/expr2
%	Modulo	expr1%expr2

Operateurs de comparaison

Operateur	Fonction
==	Égalité
!=	Inégalité
<	Inferieur strict
<=	Inferieur ou égal
>	Superieur strict
>=	Inferieur ou égal

Operateurs booléens

Opérateur	Fonction	usage
&&	Et(version optimisée)	expr1+expr2
||	Ou(version optimisée)	expr1_expr2
&	Et(version non optimisée)	-expr
|	Ou(version non optimisée)	expr1*expr2

Operateurs d'affectations

- Exemple :
  - int i, j;
  - i = 0;
  - j = i;
- évaluation de droite à gauche :
  - i = j = 2; // i et j valent 2
- • affectation combinée avec opération arithmétique
- +=, -= ,*=, &=
- i += 3; *équivaut à i = i+3;*

Opérateurs d'incrémentation et de décrémentation

Opérateur	Fonction
i++	Post-incrémentation
++i	Pré-incrémentation
i--	Post-décrémentation
--i	Pré-décrémentation

Opérateurs servant essentiellement à compacter les écritures :

n = i++; *équivaut à n = i;* i = i + 1;

n = ++i; *équivaut à i = i + 1;* n = i;

Priorité des operateurs

[ ]	(params)	expr++	expr--
++expr	--expr		
New	(type)expr		
*	/	%	
<	>	<=	>=
==	!=		
&	|	&&	||
==	*=	+=	/=
%=			

CHAPITRE II

INSTRUCTIONS DE CONTROLES
(Tests, Boucles)

Blocs d'intructions

- chaque instruction se termine par un ';'
- int i;
- i = 4*3;
- on peut grouper plusieurs instructions en un bloc (délimité par des accolades),
- exemple :
- int a=1, b=2;
- { // *début de bloc*
- int x=a;
- a = b;
- b = x;
- } // *fin de bloc*
- x = 3; // *ERREUR: x n'est pas connu ici*
- Remarque : la portée d'une variable va de sa déclaration jusqu'à la fin du bloc où elle est déclarée.

Instructions de contrôles

- Permettent de modifier l'ordre normal (*séquentiel)* *d'exécution des* instructions
- • exécution conditionnelle :
  - – if … else
- • cas multiples :
  - – switch
- • boucles :
  - – for
  - – while
  - – do … while

Instructions (if……else)

- **if (boolExpr) instruction;**
- **if (boolExpr) {**
  - instruction(s); // *exécuté si VRAI*
- **}**
- **else {**
  - instruction(s); // *exécuté si FAUX*
- **}**
- **if (boolExpr) {**
  - instruction(s); // *exécuté si VRAI*
- }
- **else if (boolExpr2) {**
  - instruction(s); // ...
- } **else {**
  - instruction(s); // ...
- }

Exemple (if……else)

```
public class Exemple_If_Else
{
   int final MIN = 100;
   int solde ;
   public static void main(String args [ ])
      {
       if ( solde < =MIN)
                System.out.println("solde insuffisant" ) ;
        else
                System.out.println("solde suffisant" ) ;
     }
  }
```

Instructions (Switch…case)

- **switch (expr) {**
- **case cst1:**
  - *instruction(s)*
- **break;**
- **case cst2:**
  - *instruction(s)*
- **case cst3:**
  - *instruction(s)*
- **break;**
- **default:**
  - instruction(s) si aucun des cas prévus
- **break;**
- **}**

Exemple (Switch…case)

```
public class Exemple_Switch{
int final MIN = 100;
int final Max = 1000;
int choix , solde;
public static void main(String args [ ])
  { switch(choix)
      { case 1: solde = MIN;
      System.out.println("solde insuffisant" ) ;
                  break;
        case 2: solde  = MAX ;
       System.out.println("solde suffisant" ) ;
                   break;
        default :  break
       }  }}
```

Boucles de répétitions(While)

- **while (boolExpr) {**
- *// corps de la boucle :*
  - instruction(s);
- **}**

ou alors

- *// cas avec une et une seule instruction*
- **while (boolExpr)**
  - instruction;

Exemple (While)

```
public class Exemple_While{
public static void main(String args [ ])
  {  while ( n < = 0)
     {  System.out.println("saisir un entier strictement positif " )
;
        Scanner sc=new Scanner(System.in);
         int n=sc.nexInt();

        if ( n < 0)  System.out.println("la saisie est invalidée:
recommencez" ) ;
     }
     }
}
```

Boucles de répétitions(do...while)

- **do {**

// corps de la boucle exécuté au moins une fois :

  - instruction(s);

- **} while (boolExpr);**

ou alors

- *// cas avec une et une seule instruction*
- **do instruction;**
- **while (boolExpr);**

Exemple(do...while)

```
public class Exemple_Do_While{
public static void main(String args [ ])
  { do
      {  System.out.println("saisir un entier strictement   positif " );
Scanner sc=new Scanner(System.in);
    int n=sc.nexInt();
if ( n < 0)  System.out.println("la saisie est invalidée: recommencez" ) ;
     }
   while ( (n < =0 );
  }
}
```

Boucles de répétitions(for)

- **for (initialisations ; boolExpr ; incr) {**
- instruction(s); // *corps de la boucle*
- **}**

équivalent à :

- initialisations;
- **while (boolExpr) {**
- instruction(s);
- incréments;
- }

Exemple(for)

```
public class Exemple_For{
public static void main(String args [ ])
  {  int tab [ ] = new int [ 100] ;  // tableau d'entiers de taille 100
    for( int i = 0 ; i < 100 ; i ++ )
      {
          tab [ i ] = i + 1;
      }
    }
}
```

Branchement inconditionnel (break)

- Elle sert à interrompre le déroulement de la boucle, en passant à l'instruction suivant la boucle.

```
public class Exemple_Break{
public static void main(String args [ ])
  {  int tab [ ] = new  int [ 10] ;  // tableau d'entiers de taille 10
    for( int i = 0 ; i < 10 ; i ++ )
      {  if ( i = = 5 ) break ;// initialiser seulement les 5 premiers elts du tableau
          tab [ i ] = i + 1;
      }     // ← le break nous branche à la sortie du for pour continuer
      for( int i = 0 ; i < 10 ; i ++ )
          System.out.println(" éléments du tableau:" +"   "+tab [ i] );
   }
}
```

Branchement inconditionnel (continue)

- L'instruction continue permet de passer prématurément au tour de boucle suivant.

```
public class Exemple_Continue{
public static void main(String args [ ])
  {  int tab [ ] = new  int [ 10] ;  // tableau d'entiers de taille 10
    for( int i = 0 ; i < 10 ; i ++ ) // ← ici
      {  if ( i = = 5 ) continue ;// on poursuit la boucle for
        tab [ i ] = i + 1;
      }
      for( int i = 0 ; i < 10 ; i ++ )
         System.out.println(" éléments du tableau:" +"   "+tab [ i] );
   }
}
```

CHAPITRE III

PROGRAMMATION ORIENTEE OBJET (POO)

Concept de Classe

- **Une classe** est la description d'un ensemble de données et de fonctions regroupées au sein d'une même entité (appelée objet).
- On peut définir une classe comme étant aussi une description abstraite d'un objet du monde réel.
- **Objet** est défini par
  - une identité
  - un état
  - un comportement
- Les fonctions qui agissent sur les données de l'objet sont aussi appelées **méthodes**.

Concept de Classe

La méthode est accessible depuis n'importe quel programme.

Les champs x et y ne sont visibles qu'à L'intérieur de la classe principe de l'encapsulation des données. Les données ne seront accessibles que par l'intermédiaire de méthodes prévues à cet effet.

```
public class Point
{
    private int x ; // champ x d'un objet Point
    private int y ; // champ y d'un objet Point
   public void initialise ( int abs, int ord )
    { x = abs ;
      y = ord ;
     }
}
```

Une méthode qui ne fournit aucun résultat:
→ void

Permet d'attribuer des valeurs initiales aux champs de l'objet.

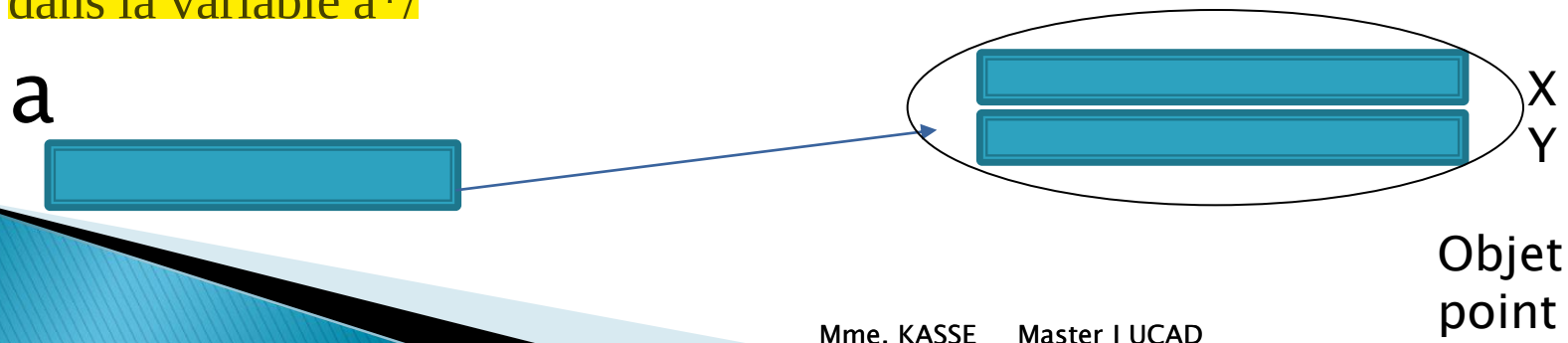
Remarque

- Une méthode peut être déclarée private : dans ce cas elle n'est visible qu'à l'intérieur de la classe où elle est définie. Pour pouvoir l'utiliser dans un autre programme, il faut nécessairement passer par une méthode publique de sa classe ayant l'appelée.

- Il est fortement déconseillé de déclarer des champs avec l'attribut public , cela nuit à l'encapsulation des données.

Création d'objet

- int a = 10 ;
- float x ;
- Point a ; /* cette déclaration ne réserve pas d'emplacement pour un objet de type Point mais simplement une référence à un objet de type Point. */
- La création d'un objet (on parle d'instanciation) se fait avec l'opérateur new :
- new Point ( ) ; /*crée un emplacement pour un objet de type Point et fournit sa référence comme résultat*/
- on pourra écrire :
- a= new Point ( ) ; /*crée un objet de type Point et on met sa référence dans la variable a*/



Manipulation d'objet

- Une fois qu'une référence à un objet est convenablement initialisée, on peut appliquer n'importe quelle méthode à l'objet correspondant.

```
a.initialise ( 2,4 ) ;
```

Operateur d'accès

Programme de test

```
class  Point01
{
    private int x ;
    private int y ;
    public void initialise ( int abs, int ord )
    { x = abs ;
      y = ord ;
     }
}
public class  TestPoint01
 { public static void main( String args [ ] )
    { Point01 a = new Point01 ( ) ;
         a.initialise( 10,12 ) ;
    }
}
```

Contructeur

En Java, la création d'objet se fait par allocation dynamique grâce à l'opérateur new qui appelle une méthode particulière : le constructeur.
En fait un constructeur permet d'automatiser l'initialisation d'un objet.

Un constructeur est une méthode qui porte le même nom que le nom de la classe et qui est sans valeur de retour. Il peut disposer d'un nombre quelconque d'arguments.

Constructeur

```
class  Point02
{
    private int x ;
    private int y ;
    public Point02 ( int abs, int ord )
     { x = abs ;
       y = ord ;
      }
   public void deplace (int dx, int dy )
     {  x += dx ;    y += dy ;
     }
   public void affiche( )
   { System.out.println("Point de coordonnées" + x + " et   "  +y);
   }
}
```

Constructeur

```
public class  TestPoint02
 { public static void main( String args [ ] )
     { Point02 a = new Point02 ( 10, 12 ) ;
      a.affiche( ) ;
              a.deplace( 10,12 ) ;
              a.affiche( ) ;
} /
```

Règles sur les constructeurs

- Une classe peut disposer de plusieurs constructeurs: ils se différencieront par le nombre et le type de leurs arguments.

- Une classe peut ne pas disposer de constructeur; dans ce cas on peut instancier des objets comme s'il existait un constructeur par défaut sans arguments et ne faisant rien.

- Mais dès qu'une classe possède au moins un constructeur, ce constructeur par défaut ne peut plus être utilisé, dans le dernier exemple on ne pouvait pas faire :
- Point a = new Point ( ) ;// incorrect s'il y'a un constructeur

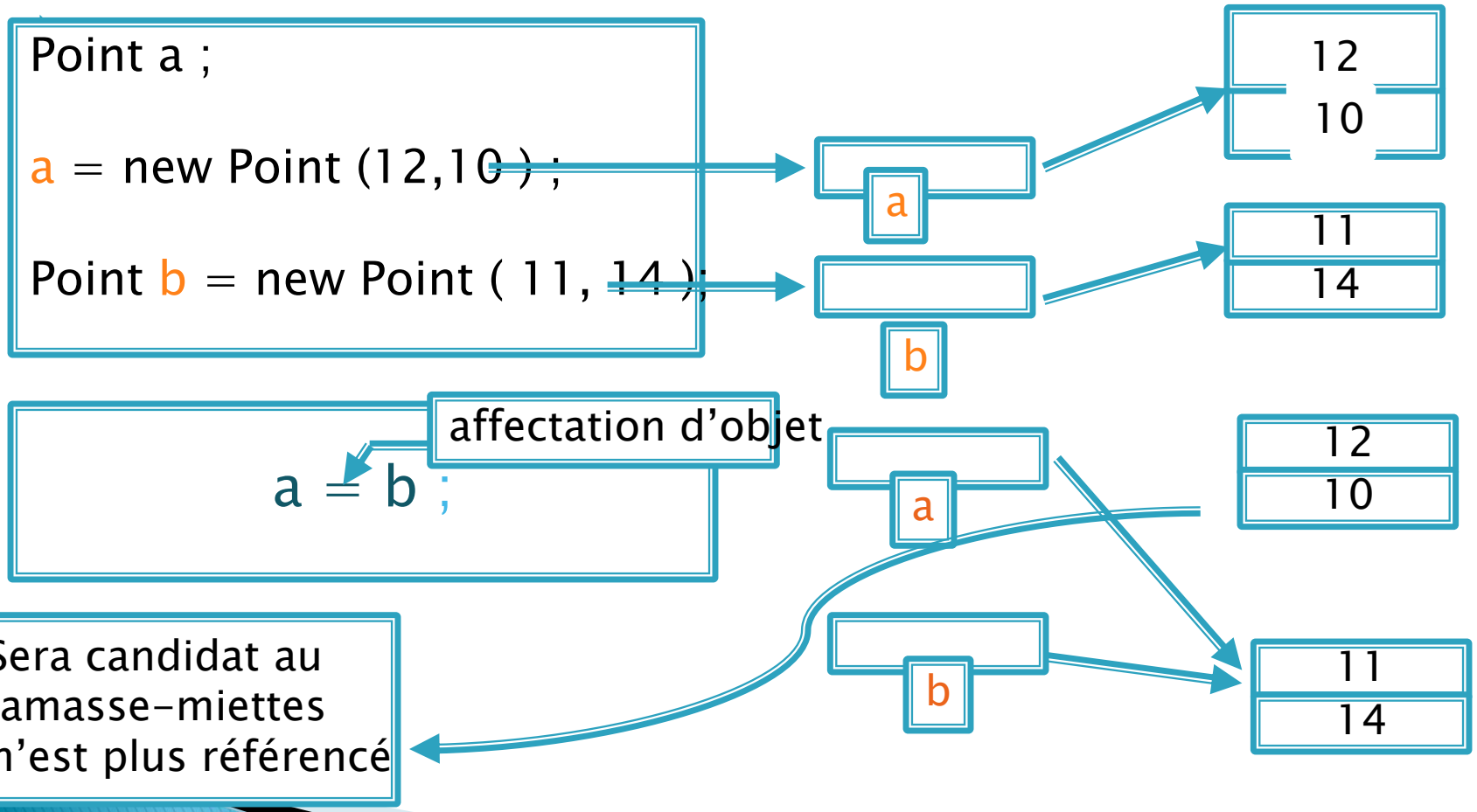
- Une classe peut disposer d'un constructeur sans arguments qui est bien différent du constructeur par défaut (appelé souvent pseudo-constructeur).

- Un constructeur peut appeler un autre constructeur de la même classe (A VOIR).

- Un constructeur peut être déclaré public ou privé.

Affectation et comparaison d'objets

.Affectation



Affectation et comparaison d'objets

- **Comparaison**

L'opérateur d'égalité = = , détermine si deux objets String spécifiés ont la même référence et non la même valeur, il ne se comporte pas en Java comme sur des éléments de type de base (int, char,...)

String a , b ;

(a = = b) renvoie **true** si les variables a et b référencent chacune le même objet de chaîne sinon il renvoie **false.**

La méthode boolean equals(Object s) teste si deux chaînes n'ayant pas la même référence ont la même valeur.

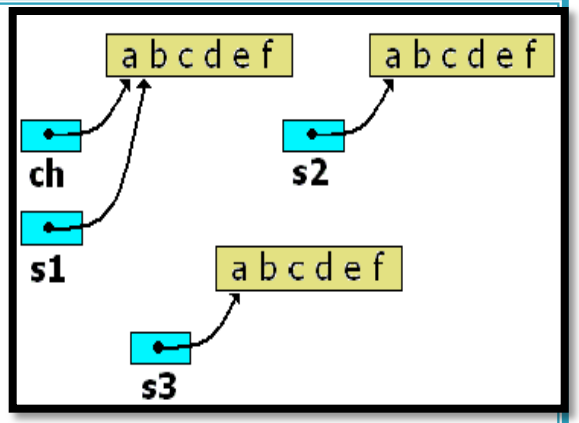
String a , b ;

a.equals (b) renvoie **true** si les variables a et b ont la même valeur sinon il renvoie false.

En d'autres termes si nous avons deux variables de String ch1 et ch2, que nous ayons écrit ch1 ="abcdef"; et plus loin ch2 = "abcdef"; les variables ch1 et ch2 n'ont pas la même référence mais ont la même valeur (valeur = "abcdef").

Affectation et comparaison d'objets

- Exemple
- String s1,s2,s3,ch;
- ch = "abcdef";
- s1 = ch;
- s2 = "abcdef";
- s3 = new String("abcdef".toCharArray( ));
- if(s1 == ch) System.out.println ("s1=ch");
- else System.out.println ("s1<>ch");
- if(s1 == s3) System.out.println ("s1=s3");
- else System.out.println ("s1<>s3");
- if(s1.equals(s2)) System.out.println ("s1 même val. que s2");
- else System.out.println ("s1 différent de s2");
- if(s1.equals(s3)) System.out.println ("s1 même val. que s3");
- else System.out.println ("s1 différent de s3");
- if(s1.equals(ch)) System.out.println ("s1 même val. que ch");
- else System.out.println ("s1 différent de ch");

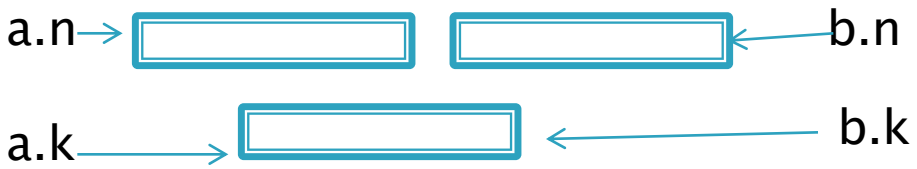


Champs et méthodes de classe: le mot clé static .

Les champs de classe ou champs statiques existent en un seul exemplaire Pour toutes les instances de la classe. On les déclare avec le mot clé static .

```
public class ChampStatic
{
    int n  ;
    static int k  ;
}
```

```
ChampStatic a = new ChampStatic( ) ;
ChampStatic b = new ChampStatic( ) ;
```



a.k et b.k peuvent être remplacés par ChampStatic.k .

Exemple

```
public class ChampStatic {
  private  static int nb;
  public ChampStatic()
    {     nb++;
    }
  public void affiche()
  {
        System.out.println ("nombre d'objets créés :" + nb  );
  }
  public static void main (String args[])
    {     ChampStatic a = new ChampStatic ( );
          a.affiche ( );
          ChampStatic b = new ChampStatic ( );
           b.affiche ( );
    }
}
```

Méthode static

- Une méthode d'une classe ayant un rôle indépendant de toute
- instance de la classe doit être déclarée avec le mot clé static
- et elle ne pourra être appliquée à aucun objet de cette classe,
- contrairement aux méthodes d'instances.
- **L'appel de la méthode ne nécessitera que le nom que de la classe.**

Une méthode statique ne peut pas agir sur des champs usuels, c'est-à-dire non statiques.

Exemple

```
public class MethodeStatic {
  private   long n;
  private  static long nombreInstanceCrees;
  public MethodeStatic( long  k)
     {   nombreInstanceCrees++ ;
         n = k ;
     }
  public void affiche ( )
  {
    System.out.println ("nombre d'objets créés :" + nombreObjet( )   );
  }
  public static long   nombreObjet( )
     {      return      nombreInstanceCrees;
     }
}
```

Surdéfinition de méthodes

- La surdéfinition de méthodes signifie qu'un même nom de méthode peut être utilisé plusieurs fois dans une même classe . Dans ce cas, **le nombre et/ou le type des arguments** doit nécessairement changé.

Exemple1

```
public class Surdef1{
  private int x ,y;
  public Surdef1 (int abs, int ord )
{ x=abs;    y=ord;     }
  public void deplace (int dx, byte
dy)
{ x += dx ; y += dy ;    }
  public void deplace (byte dx ,int
dy)
{ x += dx ;}
 public void affiche()
{
System.out.println(" Point de
coordonnees :"+ x+ " "+y);}
```

```
public static void main(String[]
args) {
   Surdef1 ex = new
Surdef1(10,10);
   ex.deplace ( 10 );
   ex.affiche ( );
   ex.deplace( 10, 10 );
   ex.affiche( );
  }
}
```

Exemple2(En cas d'embiguïté)

```
public class Surdef1{
  private int x ,y;
  public Surdef1 (int abs, int ord )
{ x=abs;    y=ord;     }
  public void deplace (int dx, byte
dy)
{ x += dx ; y += dy ;    }
  public void deplace (byte dx ,int
dy)
{ x += dx ;}
 public void affiche()
{
System.out.println(" Point de
coordonnees :"+ x+ " "+y);}
```

```
public static void main(String[] args)
{
   Surdef1 ex = new Surdef1(10,10);
int n; byte b;
   ex.deplace (n,b  );
   ex.affiche ( );
   ex.deplace( b,n );
   ex.affiche( );
   ex.deplace(b,b);
  }
}
```

Surdéfinition de constructeurs

```
Public class  Point
{     private int x, y ;
 public Point(  )
{x=0;y=0;}
 public Point( int abs )
{x=y=abs;}
    public Point( int abs, int ord )
     { x = abs ; y = ord ; }
   public void deplace (int dx, int dy )
     {  x += dx ;    y += dy ;
     }
   public void affiche( )
   { System.out.println("Point de
coordonnées" + x + " et   "  +y);
   }
}
```

```
 public static void main (String
args[])
{ Point a=new Point();
      a.affiche();
{ Point b=new Point(5);
      b.affiche();

{ Point c=new Point(3,9);
      c.affiche();
```

Transfert d'informations avec les méthodes

- Cette transmission a des conséquences totalement différentes, selon que l'on manipule des types primitifs ou bien des types objet (c-à-d des références).

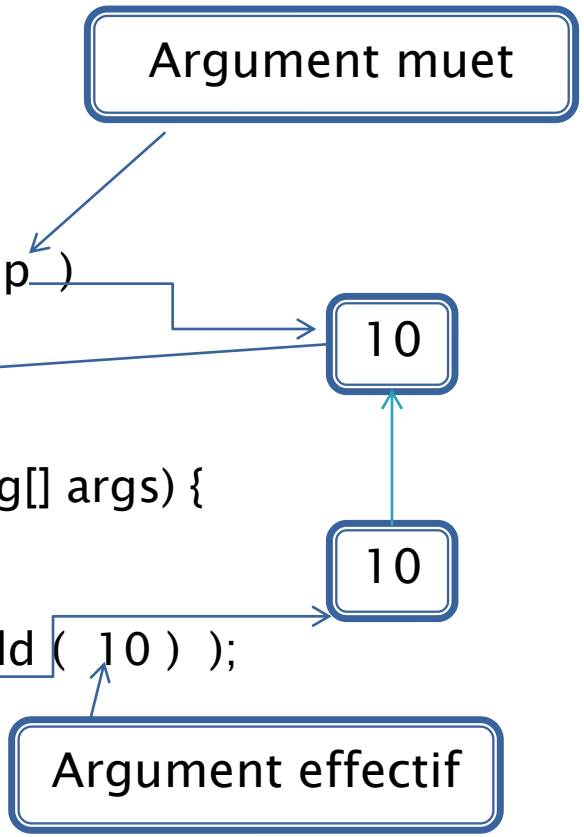
Transmission par valeur : la méthode reçoit une copie de la valeur de l'argument effectif sur laquelle elle travaille. Pas de possibilité d'agir sur la valeur de l'argument effectif.
Transmission par référence (par adresse): la méthode reçoit l'adresse de l'argument effectif sur laquelle elle travaille directement. Ici, possibilité de modifier donc la valeur de l'argument effectif.

Mécanisme de transmission

```
public class TrV {
  private double res;
 public TrV(double n) {
    res  =   n;
  }
  public double add ( double p  )
  {
      return res  +=  p;
  }
 public static void main(String[] args) {
TrV tv = new TrV(12);

     System.out.println ( tv.add (  10 )  );
  }

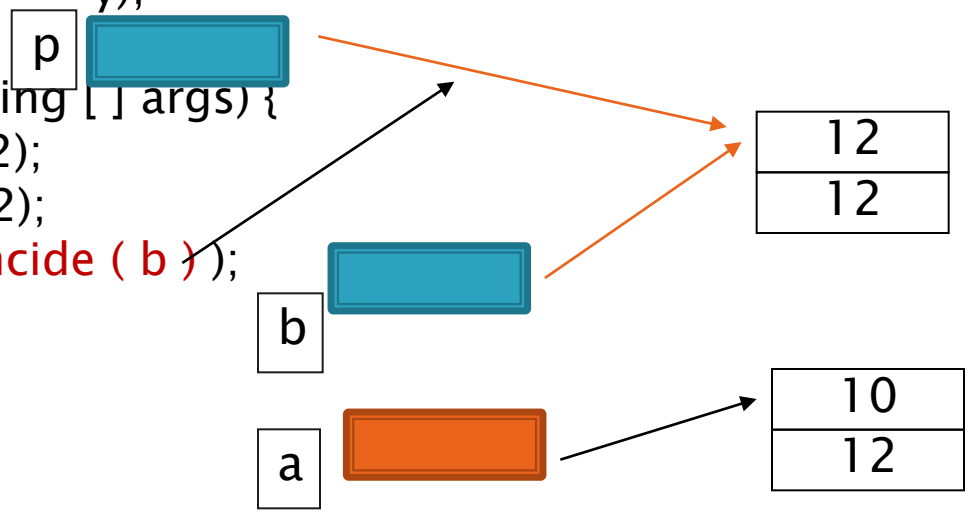
}
```



Mécanisme de transmission

```
public class Point {
  private int x;
  private int y;
  public Point (int abs, int ord) {
    x = abs;
    y = ord;
  }
  public boolean  coincide ( Point p)
  {
    return ( p.x == x &&  p.y == y);
  }
  public static void main (String [ ] args) {
    Point a = new Point(10,12);
    Point b = new Point(12,12);
    System.out.println( a.coincide ( b ) );
  }}
```

Référence à un objet de type Point transmise en argument d'une Méthode.



Autoréférence : this

Dans un appel de la forme a.coincide(b), la méthode reçoit une information lui permettant d'identifier l'objet ayant fait l'appel. Si l'information est transmise, la méthode pourra agir sur les champs spécifiques de l'objet.
Cette transmission est gérée automatiquement par le compilateur.
Mais on peut vouloir agir sur l'objet globalement au lieu d'agir sur les champs.
Dans ce cas Java utilise le mot clé this. Dans l'exemple précédent, on peut remplacer la méthode coincide par celle-ci:

```
public boolean  coincide ( Point p)
  {  // this reference ici l'objet courant ie celui ayant fait l'appel de coincide
    return ( p.x == this.x &&  p.y == this.y);
  }
```

Dans un appel a.coincide (b) a désigne l'objet courant.

Autoréférence : this

L'utilisation de this est très pratique dans l'écriture des méthodes et surtout des constructeurs.

```
public Point ( int abs, int ord ) {
        x = abs;
        y = ord;
    }
```



```
public Point ( int x, int y ) {
        this.x = x;
        this.y = y;
    }
```

Les packages

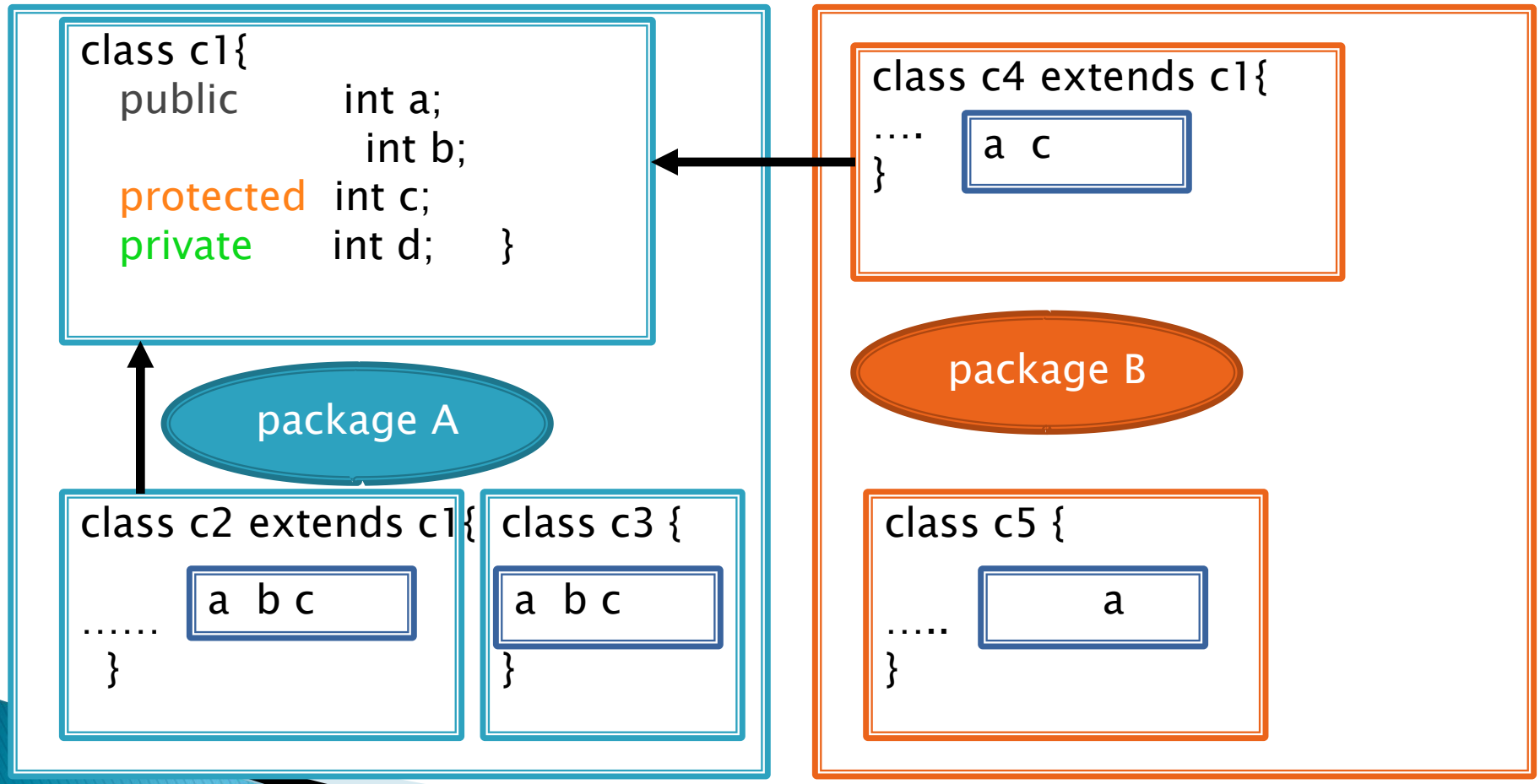
- Un package regroupe un ensemble de classes sous un même espace de nommage.
- Les noms des packages suivent le schéma : name.subname…
- Une classe Watch appartenant au package time.clock doit se trouver
- obligatoirement dans le fichier time/clock/Watch.class.
- Les packages permettent au compilateur et à la JVM de localiser les fichiers
- Contenant les classes à charger.
- L'instruction package indique à quel paquetage appartient la ou les classe(s) de
- l'unité de compilation (le fichier).
- En dehors du package, les noms des classes sont: packageName.className.
- L'instruction import packageName permet d'utiliser des classes sans les
- préfixer par leur nom de package.
- Les API sont organisées en package (java.lang, java.io, javax.swing,….)

Droits d'accès et paquetage

- En Java, il y a quatre types de droits aux méthodes et aux champs d'un objet d'une classe. Autrement dit, la portée de la visibilité des méthodes et champs est assurée par les mots clés: private, protected, vide (droit de paquetage), et public.

- Nous décrivons ici, la notion de droit d'accès parallèlement à la notion de paquetage.
- Nous séparerons la visibilité des champs et celle des méthodes.

Paquetage et encapsulation



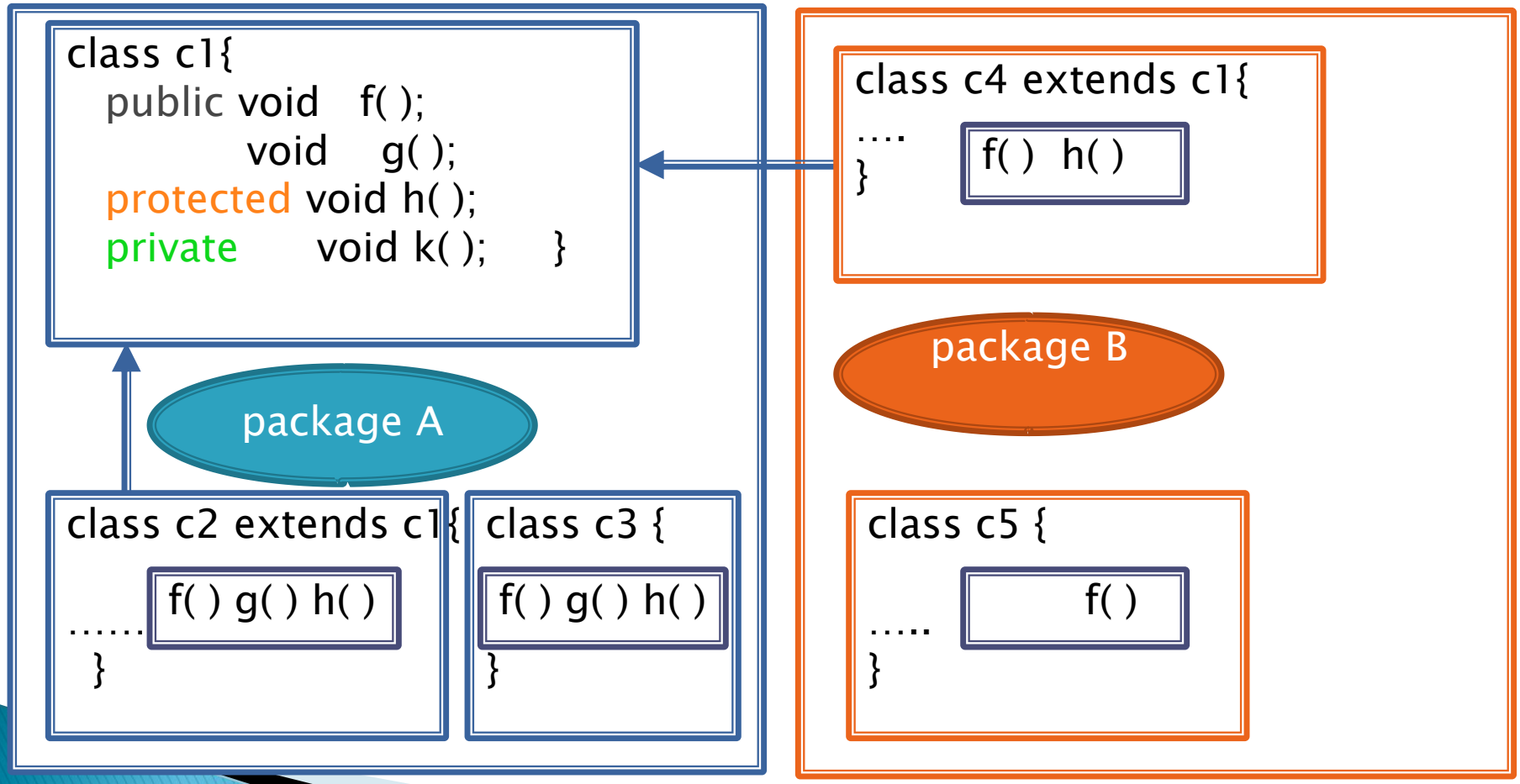
Accés au membre

```
package graph.2D;
 public class Circle{
…..
}
```

```
package graph.3D;
 public class Sphere{
…..
}
```

```
package testpackage;
 import graph.2D.*;
 public class MainClass{
public static void main (String args []){
  graph.2D.Circle c1= new graph.2D.Circle(50);
  Circle c2 = new Circle (80);
  graph.3D.Sphere s1 = new graph.3D.Sphere (100);
 Sphere s2 = new Sphere (50);}
```

Accés au membre



CHAPITRE IV

Les tableaux en JAVA

Introduction

- Dès que l'on travaille avec de nombreuses données homogènes (de même type), la première
- structure de base permettant le regroupement de ces données est le **tableau.** Java comme tous les langages algorithmiques propose cette structure au programmeur. Comme pour les **String,** pour des raisons d'efficacité dans l'encombrement mémoire, **les tableaux sont gérés par Java, comme des objets.**

Les tableaux Java sont comme en Delphi, des tableaux de tous types y compris des types objets.
Il n'y a pas de mot clef spécifique pour la classe tableaux, mais l'opérateur symbolique **[ ]** indique qu'une variable de type fixé est un tableau.
La taille d'un tableau doit obligatoirement avoir été définie avant que Java accepte de l'utilisiez !

Tableaux à une dimension

- Déclaration d'une variable de tableau :

- **type** identificateur [ ] ;
  - ou
- **type** [ ] identificateur ;

- **Exemples:**

- **int [ ] table1;**
- **char [ ] table2;**
- **float [ ] table3;**
- **…**
- **String [ ] tableStr;**

Définition et création d'un tableau

- Déclaration d'une variable de tableau avec définition explicite de taille :
- **int [ ] table1 = new int [5];**
- **char [ ] table2 = new char [12];**
- **float [ ] table3 = new float [8];**
- ...
- **String [ ] tableStr = new String [9];**
- Le mot clef **new correspond à la création d'un nouvel objet (un nouveau tableau) dont la taille**
- est fixée par la valeur indiquée entre les crochets. Ici 4 tableaux sont créés et prêts à être utilisés :
- table1 contiendra 5 entiers , table2 contiendra 12 caractères, table3 contiendra 8 réels en simple précision et tableStr contiendra 9 chaînes de type **String.**

Déclaration et création d'un tableau

- Les tableaux peuvent être initialisés par l'intermédiaire d'une liste de valeurs séparées par une virgule et compris entre des accolades .
- type [ ] identificateur = { valeur1, ..., valeurN };
- type identificateur [ ] = { valeur1, ..., valeurN };
- int [ ] table1 = {17,-9,4,3,57};
- char [ ] table2 = {'a','j','k','m','z'};
- float [ ] table3 = {-15.7f,75,-22.03f,3,57};
- String [ ] tableStr = {"chat","chien","souris","rat","vache"};
- Dans cette éventualité Java crée le tableau, calcule sa taille et l'initialise avec les valeurs fournies.

Déclaration et création d'un tableau

- Il existe un attribut général de la classe des
- tableaux, qui contient la taille d'un tableau
- quelque soit son type, c'est l'attribut length.
- *Exemple :*
- int [ ] table1 = {17,-9,4,3,57};
- int taille;
- taille = table1.length; *// taille = 5*

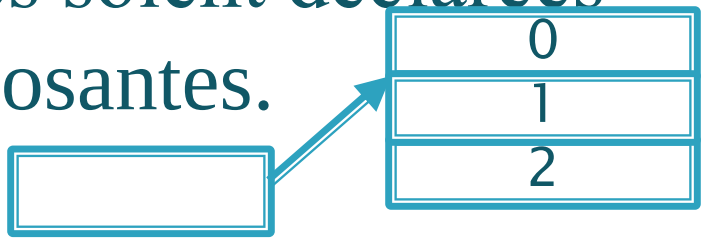
Accéder aux éléments d'un tableau

- Un tableau en Java comme dans les autres langages algorithmiques, s'utilise à travers une cellule de ce tableau repérée par un indice obligatoirement de type entier. Le premier élément d'un tableau est numéroté 0, le dernier length-1.
- On peut ranger des valeurs ou des expressions du type général du tableau dans une cellule du tableau.
- Exemple avec un tableau de type int
- int [ ] table1 = new int [5];
- table1[0] = -458;
- table1[4] = 5891;
- table1[5] = 72;
- for (int i = 0 ; i<= table1.length-1; i++)
- table1[i] = 3*i-1; //

Accéder globalement au tableau

La manipulation globale d'un tableau se fait par affectation de tableaux. Il est possible d'affecter une variable de type tableau à une autre variable, à condition qu'elles soient déclarées avec le même type de composantes.

```
int [ ] tab = new int [3] ;
for(int i = 0; i < 3; i++)   tab [i] = i ;
```




```
int [ ] t = new int [2] ;
for(int i =0 ;  i < 2 ;i++)   t[i] = i+10 ;
```

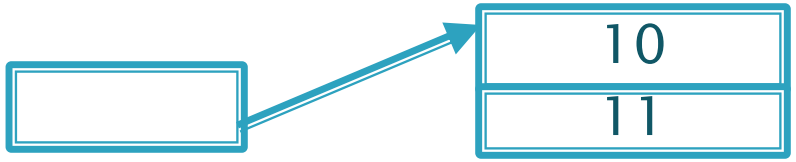


Tableau en argument ou en valeur de retour

- Comprenez la transmission d'un tableau en argument ou en valeur de retour d'une méthode comme celle réalisée avec les objets.
- Lorsqu'on transmet un nom de tableau en argument d'une méthode, on transmet en fait (une copie de)la référence au tableau .La méthode agit alors directement
- sur le tableau concerné et non sur une copie .

Exemple

```
public class TabInverse {
public static int[ ] inverseTab (int t[ ])
  { int tabres[ ] = new int [t.length];
     for ( int i =  t.length - 1; i > = 0 ; i--)
        tabres [ t.length – i - 1] = t[ i ];
     return tabres;
  }
  public static void afficheTab( int t[ ])
  { for ( int i = 0; i < t.length; i++)
    System.out.print ( t[ i ]+ " ");
  }
```

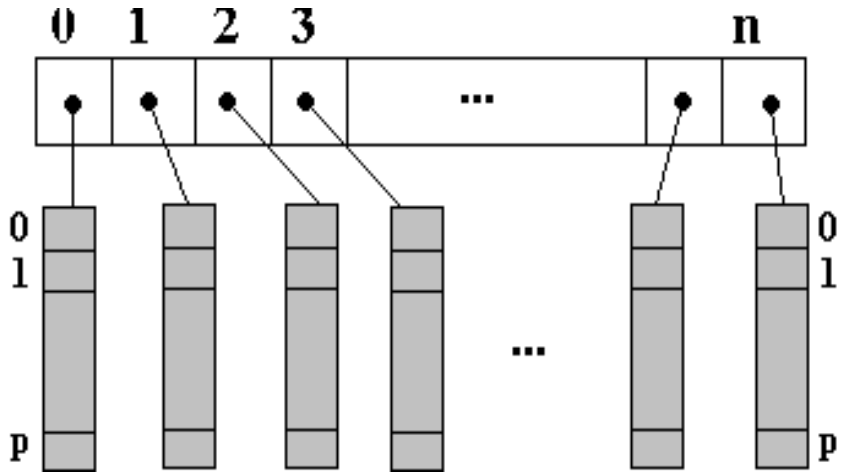
```
public class RunTabInverse {
public static void main( String args[
])
  {  int tabAinverser [ ] = new int [5];
       tabAinverser [0] = 1;
       tabAinverser [1] = 80;
       tabAinverser [2] = 71;
       tabAinverser [3] = 6;
       tabAinverser [4] = 500;
      afficheTab(inverseTab
(tabAinverser));
  }
}
```

Tableaux à deux dimensions

- Pour déclarer un tableau à deux dimensions, on peut utiliser l'une de ces trois
- déclarations qui sont équivalentes :
- int t [ ] [ ] ;
- int [ ] t [ ] ;
- int [ ] [ ]t ;
- Elles déclarent que t est une référence à un tableau, dans lequel chaque élément
- est lui-même une référence à un tableau d'entiers . Pour l'instant, aucun tableau
- de cette sorte n'existe encore .

Création de tableaux à 2 dimensions

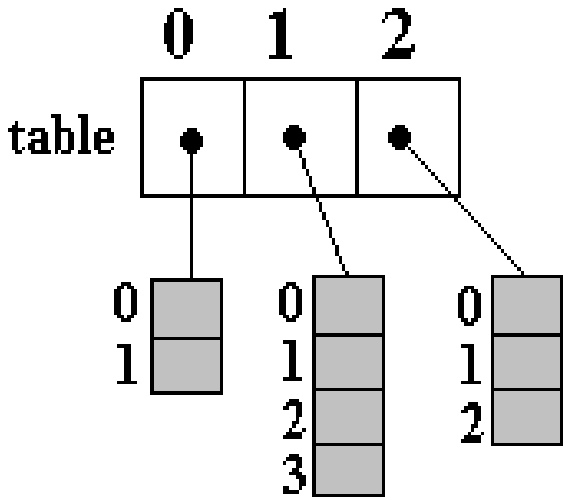
- **Exemple**
- int [ ][ ] table1 = new int [2][3];
  table1[0][0] = -458;
- int [ ][ ] table = new int [n+1][p+1];



Autres exemples

Rien n'oblige les tableaux référencés d'avoir la même dimension, ce type de tableau se dénomme tableaux en escalier ou tableaux déchiquetés en Java :

- **int [ ][ ] table = new int [3][ ];**
- table[0] = **new int[2];**
- table[1] = **new int[4];**
- table[2] = **new int[3];**



Taille d'un tableau multi indices

- Le même attribut général length de la classe des tableaux, contient **la taille du tableau :**
- ***Exemple : matrice à deux lignes et à 3 colonnes***
- **int [ ][ ] table1 = new int [2][3];**
- **int taille;**
- taille = table1.length;
- taille = table1[0].length;
- taille = table1[1].length;
- Java initialise les tableaux par défaut à 0 pour les **int, byte, ... et à null pour les objets**

Les chaines de caractères(String)

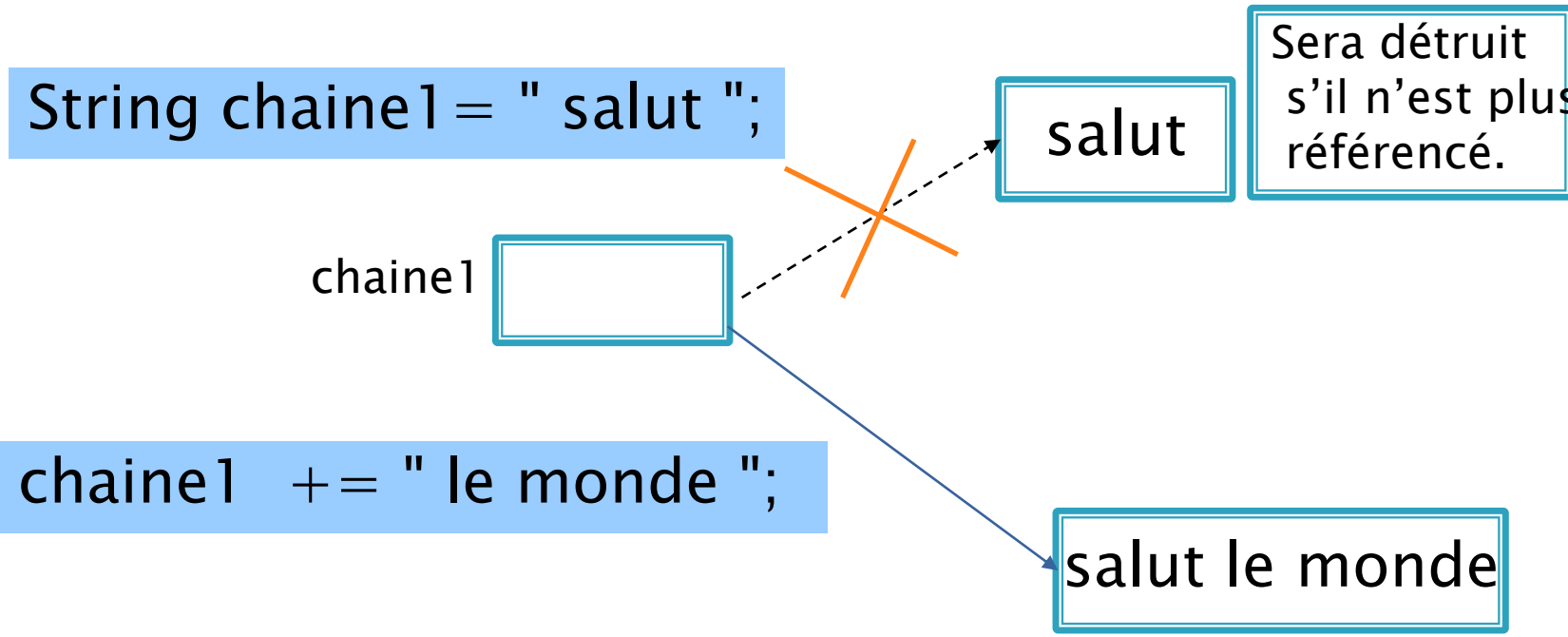
La classe **String** permet de manipuler « les chaînes de caractères ».

String chaine;
chaine =« "ca c 'est un objet chaine de caractère »

La classe String possède plusieurs constructeurs dont:
String( ); // pour construire une chaine vide
String (String original); // crée un objet contenant la chaine original

Les chaines de caractères(String)

Un objet de type String n'est pas modifiable.



Les méthodes de la classe String (1 /4)

```
/*retourne la longueur de l'objet String.*/
int length ( )
/*retourne un nouvel objet String résultant du remplacement de toutes les
occurrences d'un caractère donnée par un autre caractère. */
String replace(char oldChar, char newChar)
/*remplace dans une chaîne de caractères, chaque sous-chaîne qui correspondent
 à l'expression régulière fournie, par une chaîne de caractères de remplacement*/
String replaceAll (String origine, String remplacement)
/*teste si l'objet String démarre au préfixe spécifié. */
boolean startsWith (String prefix)
/*retourne une nouvelle chaîne de caractères qui est une sous-chaîne de l'objet String
 par l'intermédiaire d'un intervalle commençant à l'index spécifié jusqu'à la fin. */
String substring (int beginIndex)
/*retourne une nouvelle chaîne de caractères qui est une sous-chaîne de l'objet String
 par l'intermédiaire d'un intervalle spécifié */
String substring(int beginIndex, int endIndex)
```

Les méthodes de la classe String (2/4)

```
/*retourne le caractère positionné à l'index spécifié. */
char charAt (int index)
/*compare l'objet String à un autre objet o. Retourne 0 en cas d'égalité
-1 en cas d'infériorité et 1 en cas de supériorité */
int compareTo (Object o)
/*comparaison lexicographique de deux chaines. Retourne 0 en cas
d'égalité -1 en cas d'infériorité et 1 en cas de supériorité */
int compareTo(String anotherString)
/*compare deux chaînes lexicographiquement en ignorant la casse de
caractères*/
int compareToIgnoreCase (String str)
/*concatène l'objet String à une autre chaîne de caractères. */
String concat (String str)
/*retourne true si et seulement si l'objet String représente la même
séquence de caractères comme l'objet StringBuffer spécifié. */
boolean contentEquals (StringBuffer buff)
```

Les méthodes de la classe String (3/4)

```
/*retourne un nouvel objet String qui représente la séquence de caractères dans le tableau spécifié. */
static String copyValueOf (char[ ] data)
/*teste si la fin de l'objet String correspond au suffixe spécifié. */
 boolean endsWith (String suffix)
/*compare le contenu de deux chaînes de caractères entre elles. */
boolean equals (Object anObject)
/*compare l'objet String à un autre objet de même type en ignorant la casse de caractères. */
boolean equalsIgnoreCase (String anotherString)
/*retourne l'index à l'intérieur de l'objet String de la première occurrence du caractère spécifié */
int indexOf (int ch)
/*retourne l'index à l'intérieur de l'objet String de la première occurrence du caractère spécifié à partir d'une certaine position */
int indexOf(int ch, int fromIndex)
```

Les méthodes de la classe String (4/4)

```
/*retourne l'index à l'intérieur de l'objet String de la première occurrence de
 la sous-chaîne spécifiée. */
int indexOf (String str)
/*retourne l'index à l'intérieur l'objet String de la première occurrence de la
 sous-chaîne spécifiée à partir d'une certaine position. */
int indexOf(String str, int fromIndex)
/*retourne l'index à l'intérieur de l'objet String de la dernière occurrence
 du caractère spécifié */
int lastIndexOf (int ch)
/*retourne une chaîne de caractères issue de l'objet String. */
String toString()
/*retourne un représentation sous forme d'un objet String de la valeur
 de type int passée en argument */
static String valueOf (int i)
```

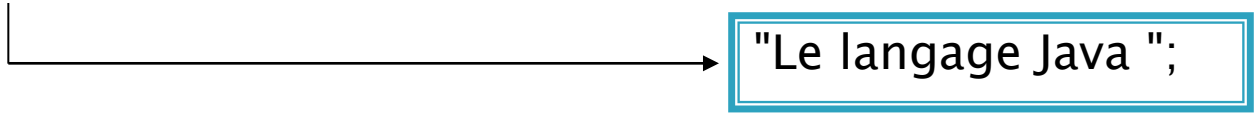
Utilisation de quelques méthodes

String ch = "Le langage Java est vraiment très puissant";

ch.length ( );// longueur de la chaine est 42 (espaces compris)
ch.substring (11);



ch.substring (0,16);//attention: le caractère à l'indice 16 n'est pas extrait



ch.toUpperCase ( );// il existe aussi toLowerCase



ch = " bonjour \n" ;
ch.trim( ) //supprime les espaces de début et de fin dans la chaine

→ "bonjour"

Utilisation de quelques méthodes

```
String s= " java " ;
String ch = " java " ;
s.equals (ch); // ici renvoie true car equals est redéfinie dans String

s.charAt (0); // renvoie le caractère à la position 0 donc j

char c [ ] = s.toCharArray( );// renvoie un tableau de caractères
```

→ c vaut { 'j','a','v','a'}

```
ch.indexOf ( a); // l'indice de la 1ère occurrence trouvée
```

→ renvoie la valeur 1

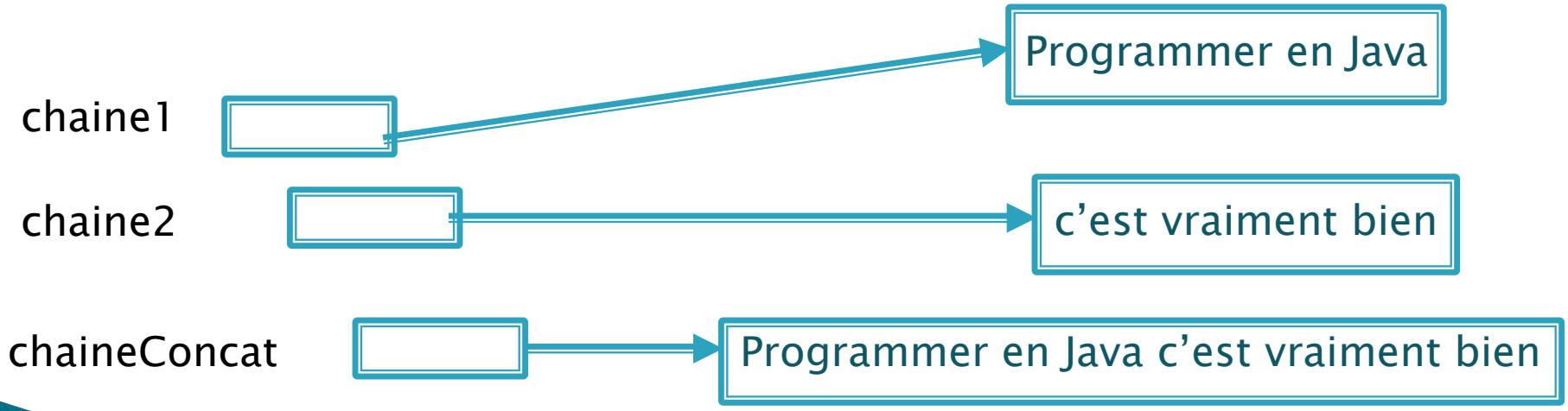
```
ch.indexOf ( a,2); // l'indice de la 1ère occurrence trouvée à partir de 2
```

→ renvoie la valeur 3

L'opérateur +

L'opérateur + permet de concaténer deux chaînes de caractères. Il est défini lorsque ses deux opérandes sont des chaînes. Il renvoie un résultat qui est la Concaténation des deux chaînes.

```
String chaine1 = "Programmer en Java" ;
String chaine2 = "c'est vraiment bien";
String chaineConcat = chaine1 + chaine2;
```



Conversion de types primitifs en chaines

La classe String possède une méthode statique valueOf surdéfinie avec un argument de chaque type primitif qui permet de convertir n'importe quel type primitif en chaîne de caractères.

```
 int n = 541;
 double d = 120.56;
String intCh = String.valueOf ( n); //intCh contient la chaine " 541"
String doubleCh = String.valueOf ( d); //doubleCh contient la chaine " 120.56 "
```

L'écriture intCh = String.valueOf (n); est équivalent à celle-ci:

```
intCh = " " +n;//on utilise une chaine vide pour recourir à
                // l' operateur + qui convertit n en String
```

Conversion de chaines en typesprimitifs

Il est possible de convertir une chaine dans un type primitif en recourant
aux classes enveloppes définies pour chaque type primitif.

```
String ch = " 3521 " ;
int n = Integer.parseInt (ch);// n contient la valeur entière 3521
```

On dispose aussi des méthodes:

Byte.parseByte
Short.parseShort
Integer.parseInt
Long.parseLong
Double.parseDouble
Float.parseFloat

Chaines et tableaux

En Java, on peut convertir un tableau en chaîne et vice versa:

```
char [ ] mot = { ‘b’,’o’,’n’,’j’,’o’,’u’,’r’};
/*on construit une chaine à partir d’un tableau de caracteres*/
String ch = new String (mot); // ch = " bonjour "
/*constructeur avec le premier caractère et le nombre de
caracteres*/
String ch2 = new String (mot, 3,4);// ch = " jour "

String ch = " bonjour " ;
char [ ] mot = ch.toCharArray ( );

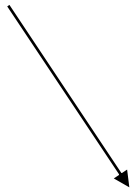
            mot = { ‘b’,’o’,’n’,’j’,’o’,’u’,’r’};
```

La classe StringTokenizer

Cette classe n' a aucun lien direct avec la classe String, elle se trouve d'ailleurs dans le paquetage java.util . Elle apporte un rôle dans la manipulation des chaînes en facilitant la division de chaînes en sous-chaînes selon un nombre de « délimiteurs ».

Cette méthode divise une chaîne en différents éléments appelés tokens.

```
String s = " Java, est: un .langage ;interessant";
StringTokenizer st = new StringTokenizer (s, " ,;:. " );
while (st.hasMoreTokens ( ))//tant qu'il y a des tokens
 { st.nextToken( ) ; // renvoie le premier element et se positionne sur le suivant
                     // cette méthode renvoie un objet String.
 }
```



On trouve 5 tockens :	Java	est	un	langage	interessant

CHAPITRE V

POO: HERITAGE

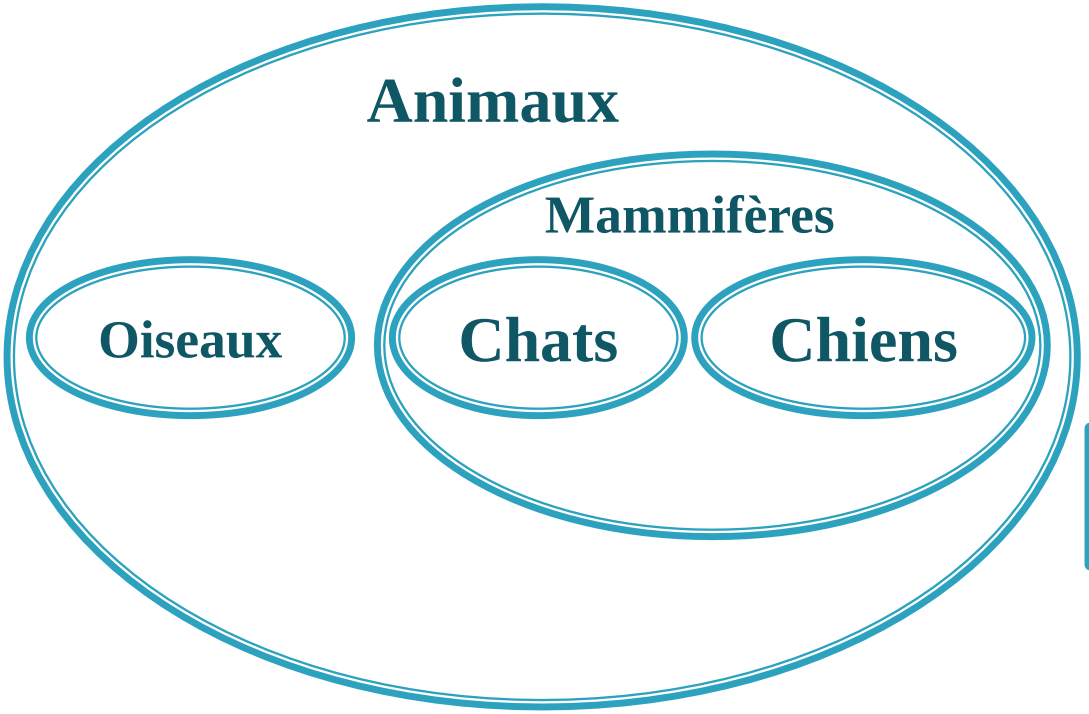
Héritage

On a parfois besoin de définir un type d'objet similaire à un type existant, avec quelques propriétés supplémentaires. L'héritage permet de définir ce nouveau type sans tout reprogrammer.

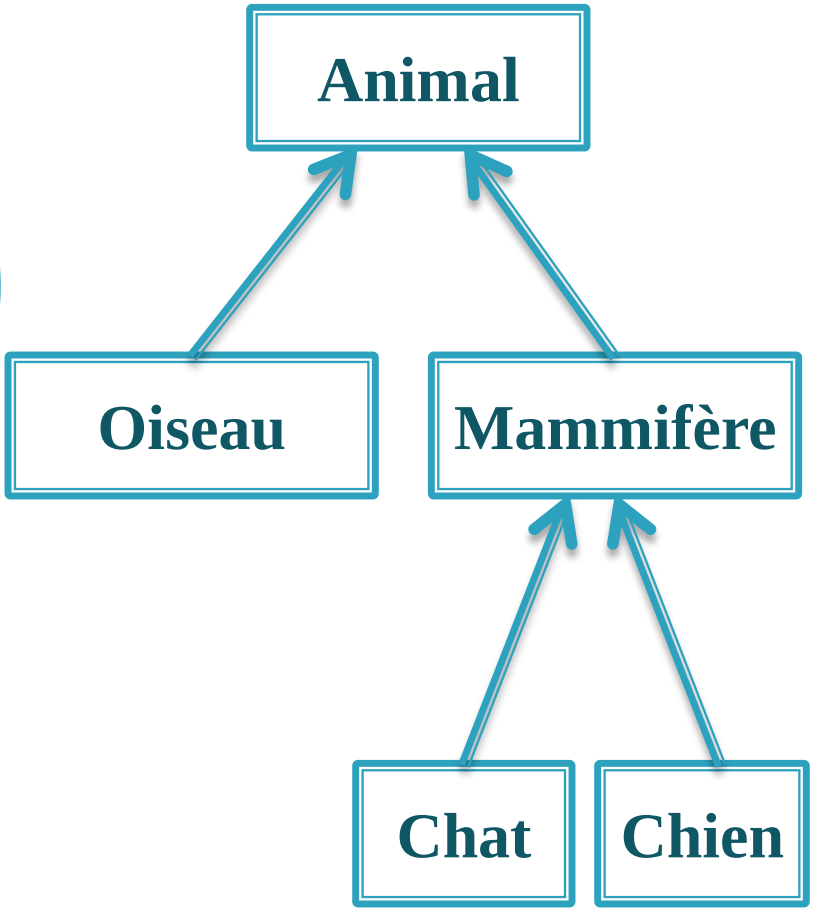
Une classe fille qui dérive (on dit qui étend en Java) d'une seule classe mère, hérite de sa classe mère toutes ses méthodes et tous ses champs(Sauf ceux déclarés privé).

Notion de hiérarchie de classes

Vision ensembliste



Vision hiérarchique



Concepte

En Java la syntaxe de l'héritage fait intervenir le mot clef **extends**,
Une déclaration du type :

```
class ClasseFille extends ClasseMere {
}
```

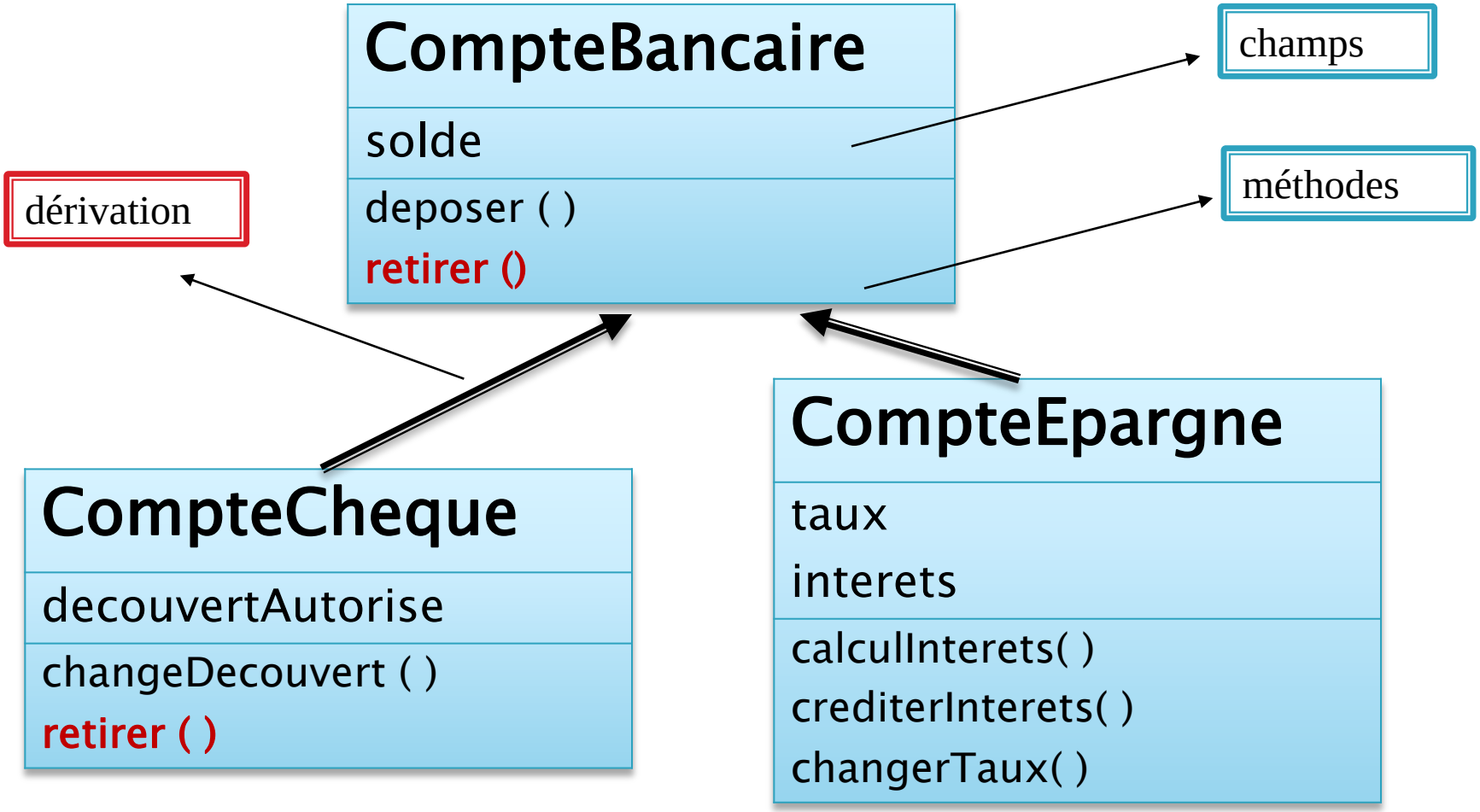
On se propose de spécialiser la gestion des comptes. On crée alors une classe **CompteChèque** et une autre classe **CompteEpargne** qui dérivent de la classe **CompteBancaire.**

```
public class CompteBancaire{…..}
```

```
public class CompteCheque  extends  CompteBancaire {…..}
```

```
public class CompteEpargne  extends  CompteBancaire {…..}
```

Concepte


CompteBancaire
solde
deposer ( )
retirer ()
champs
méthodes
dérivation
CompteCheque
decouvertAutorise
changeDecouvert ( )
retirer ( )
CompteEpargne
taux
interets
calculInterets( )
crediterInterets( )
changerTaux( )

Le mot clef SUPER

Ce mot clef **super** référence la classe mère et à travers lui, il est possible d'accéder à tous les champs et à toutes les méthodes de la super-classe (classe mère).

Au début du corps d'un constructeur de la classe dérivée, on peut appeler explicitement un constructeur de la classe de base avec les paramètres souhaités grâce à la notation : **super(*paramètres*)**

Application

Supposons disposer d'une classe CompteBancaire:

```
package 1;

public class CompteBancaire {
  double solde ;
        CompteBancaire (double solde )
                { this.solde = solde;
                }
      void deposer ( double montant)
          { solde +=montant;
          }
       void retirer (double montant)
         {  if (solde >=montant )
                  solde -=montant ;
         }
}
```

Application

```
package1;
public class CompteCheque extends CompteBancaire {
  CompteCheque (double solde, double decouvertAutorise)
                {
                  super (solde);
                   this.decouvertAutorise = decouvertAutorise;
                }
      void retirer (double montant) // methode redefinie
        {  if (solde + decouvertAutorise >=montant)
                  solde -=montant ;
        }
}
```

Application

```
package 1;

public class CompteEpargne  extends   CompteBancaire {


CompteEpargne (double solde, double taux)
                { super (solde);
                  this. taux = taux;
                }
}
```

Redéfinition de membres

```
package 1;

 class CompteBancaire {
  double solde ;
// .....
void retirer (double montant)

        {  if (solde >=montant ;
                solde -=montant ;
        }
void imprimeHistorique( )
{ System.out.print (" solde ="
+solde );
}
```

```
package 1;

class CompteCheque extends CompteBancaire {

double decouvertAutorise;
 // methode redefinie
      void retirer (double montant)

      {  if (solde + decouvertAutorise>=montant)
                  solde -=montant ;
         }
void imprimeHistoriqueCheque( )
{ System.out.print (" solde =" +solde + " " +
"decouvertAutorise=" +decouvertAutorise);
 }
}
```

Redéfinition de membres

Avec :
CompteBancaire cb; CompteCheque cc;
l'appel : cb.retirer (20000);
appelle la méthode retirer de CompteBancaire.
l'appel : cc.retirer (20000);
appelle la méthode retirer de CompteCheque.
On se base tout simplement sur le type de l'objet pour déterminer la classe de la méthode appelée.
Pour bien voir l'intérêt de la redéfinition des méthodes, examinons la Méthode imprimeHistorique de la classe **CompteBancaire** qui permet d'afficher le solde pour un compte et la méthode imprimeHistoriqueCheque de la classe **CompteCheque** qui affiche non seulement le solde (qui est un membre hérité) mais aussi le decouvertAutorise.
Dans cette dernière, il y a une information qui est déjà prise en compte dans la méthode imprimeHistorique. La situation précédente peut être améliorée de cette façon:

Redéfinition de membres

```
package 1;

 class CompteBancaire {
  double solde ;
// .....

void retirer (double montant)

 {  if (solde >=montant )
                solde -=montant ;
        }
void imprimeHistorique( )
{ System.out.print (" solde ="
+solde );
}
```

```
package 1;

class CompteCheque extends CompteBancaire
{

double decouvertAutorise;
 // methode redefinie

        void retirer (double montant)
{  if (solde + decouvertAutorise >=montant )
                solde -=montant ;
        }
void imprimeHistoriqueCheque( )
{
        imprimeHistorique( );
System.out.print ("  et le decouvertAutorise ="
 +decouvertAutorise);
 }}
```

Redéfinition de membres

```
package1;

 class CompteBancaire {
  double solde ;
// .....
void retirer (double montant)
        {  if (solde >=montant ;
              solde -=montant ;
        }
void imprimeHistorique( )
{ System.out.print (" solde ="
+solde );
}
```

super obligatoire

```
package1;

class CompteCheque extends CompteBancaire {

double decouvertAutorise;
 // methode redefinie
     void retirer (double montant)
{  if (solde + decouvertAutorise >=montant ;
                  solde -=montant ;
        }
{    void imprimeHistorique( )
        super.imprimeHistorique( );
System.out.print (" et le decouvertAutorise ="
 +decouvertAutorise);
 }
}
```

Surdéfinition et héritage

```
package  1;

 class Calcul {
 public double division ( int a)
   {   // instructions
   }
}

class Calcul11 exends Calcul {
 public float division ( float a)

   {   // instructions
   }
}
```

```
Calcul c; Calcul11 c11;
int n; float p;

c.division (n);

c.division (p);

c11.division (n);

c11.division (p);
```

La recherche d'une méthode acceptable ne se fait qu'en remontant la hiérarchie d'héritage, jamais en la descendant. C'est pourquoi l'appel c.division (p); ne peut être satisfait.

Contraintes sur la redéfinition

En cas de redéfinition, Java impose seulement l'identité des signatures mais aussi du type de la valeur de retour

```
package    allndong.compte;

 class CompteBancaire {
  double solde ;
 //
 void  retirer (double montant)

        {  if (solde >=montant ;
                solde -=montant ;
        } }
```

```
package    allndong.compte;

class CompteCheque extends CompteBancaire
{
   double decouvertAutorise;
  // methode redefinie
   void  retirer (double montant)

{  if (solde + decouvertAutorise >=montant ;
                solde -=montant ;

 }
}
```

Valeurs de retour identiques

Signatures identiques

Contraintes sur la redéfinition

La redéfinition d'une méthode ne doit pas diminuer les droits d'accès à cette méthode.
Par contre, elle peut les augmenter.

```
class A
{ public void f ( int n){……}
}
class B extends A
{// impossible de mettre private
private void f ( int n) {….}
}
```

```
class A
{ private void f ( int n){……}
}
class B extends A
{// augmente les droits d acces: possible
public void f ( int n) {….}
}
```

Règle sur la Redéfinition et la surdéfinition

Si une méthode d'une clase dérivée a la même signature qu'une méthode d'une classe ascendante :

- les valeurs de retour des deux méthodes doivent être exactement de même type,
- le droit d'accès de la méthode de la classe dérivée ne doit pas être plus élevé que celui de la classe ascendante,
- la clause throws de la méthode de la classe dérivée ne doit pas mentionner des exceptions non mentionnées dans la clause throws de la méthode de la classe ascendante(la clause throws sera étudiée ultérieurement).

Si ces trois conditions sont remplies, on a affaire à une redéfinition. Sinon, il s'agit d'une erreur.

Dans les autres cas, c'est-à-dire lorsqu'une méthode d'une classe dérivée a le même nom qu'une méthode d'une classe ascendante, avec une signature différente, on a affaire à une surdéfinition .

Le Polymorphisme

Un langage orienté objet est dit polymorphique, s'il offre la possibilité de pouvoir percevoir un objet en tant qu'instance de classes variées, selon les besoins.

Le **surclassement** et la **liaison dynamique** (ligature dynamique) servent à mettre en œuvre le **polymorphisme**.

Le Surclassement

Une classe B qui hérite de la classe A peut être vue comme un sous-type du type défini par la classe A. On parle de **surclassement ou upcasting si :**« à une référence déclarée de type A il est possible d'affecter une valeur qui est une référence vers un objet de type B (surclassement ou upcasting) »

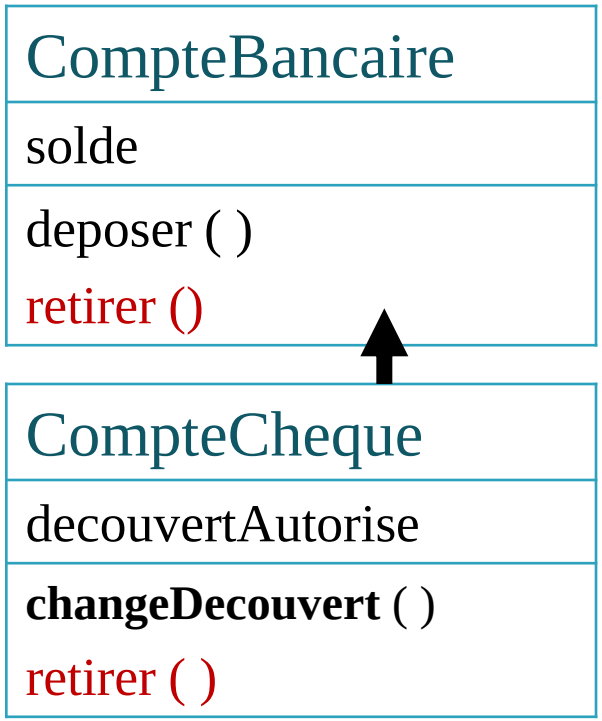
Exemple:

```
CompteBancaire cb;
cb = new CompteCheque( 100000, 50000 );
```

A la compilation

- Lorsqu'un objet est « surclassé », il est vu par le compilateur comme un objet du type de la référence utilisée pour le désigner
- Ses fonctionnalités sont alors restreintes à celles proposées par la classe du type de la référence

Exemple de Surclassement




```
CompteCheque CC = new CompteCheque(100,50);
CompteBancaire cb;
cb = CC; // surclassement
cb.retirer (50);
cc.retirer( 25);

cb.deposer (500);
cc.deposer ( 250);

 cb.changedecouvert ( );

cc.changeDecouvert( );
```

La liaison dynamique

A l'exécution

•Lorsqu'une méthode d'un objet est accédée au travers d'une référence « surclassée », c'est la méthode telle qu'elle est définie au niveau de la classe effective de l'objet qui est invoquée et exécutée. La méthode à exécuter est déterminée à l'exécution et non pas à la compilation. On parle de **lien dynamique.**

« la méthode à exécuter est déterminée à l'exécution (run– time) et non pas à la compilation ».

« la méthode définie pour le type réel de l'objet recevant le message est appelée et non pas celle définie pour son type déclaré ».

Exemple CompteBancaire cb = new CompteCheque (500,100);

Type déclaré

Type réel

Exemple

CompteBancaire

```
void retirer (double montant)
{  if (solde >=montant)
        solde -=montant ;
}
```

CompteCheque

```
void retirer (double montant)
{if (solde + decouvertAutorise >=montant ;
        solde -=montant ;
}
```

```
CompteBancaire cb = new  CompteCheque (500,100);

cb.retirer ( 50);
```

Les classes abstraites

En Java, une classe peut annoncer une méthode sans la définir. Une telle classe est dite ***abstraite*.**
Le but est d'avoir des modèles génériques permettant de définir ultérieurement des actions spécifiques.

3 règles à retenir

- Si une seule des méthodes d'une classe est abstraite, alors la classe devient aussi abstraite. Une méthode **abstraite n'est qu'une signature de méthode sans** implémentation dans la classe.
- On ne peut pas instancier une classe abstraite car au moins une de ses méthodes n'a pas d'implémentation.
- Toutes les classes filles héritant de la classe mère abstraite doivent implémenter toutes ses méthodes abstraites ou sinon elles sont aussi abstraites.

Les classes abstraites

On définit une classe abstraite en Java en utilisant le mot clé **abstract** devant le nom de la classe:

```
abstract class A {
...
abstract void P();
...
void Q() {...}
}
```

```
class B extends A {
...
void P(){...}
...
void Q() {...}
}
```

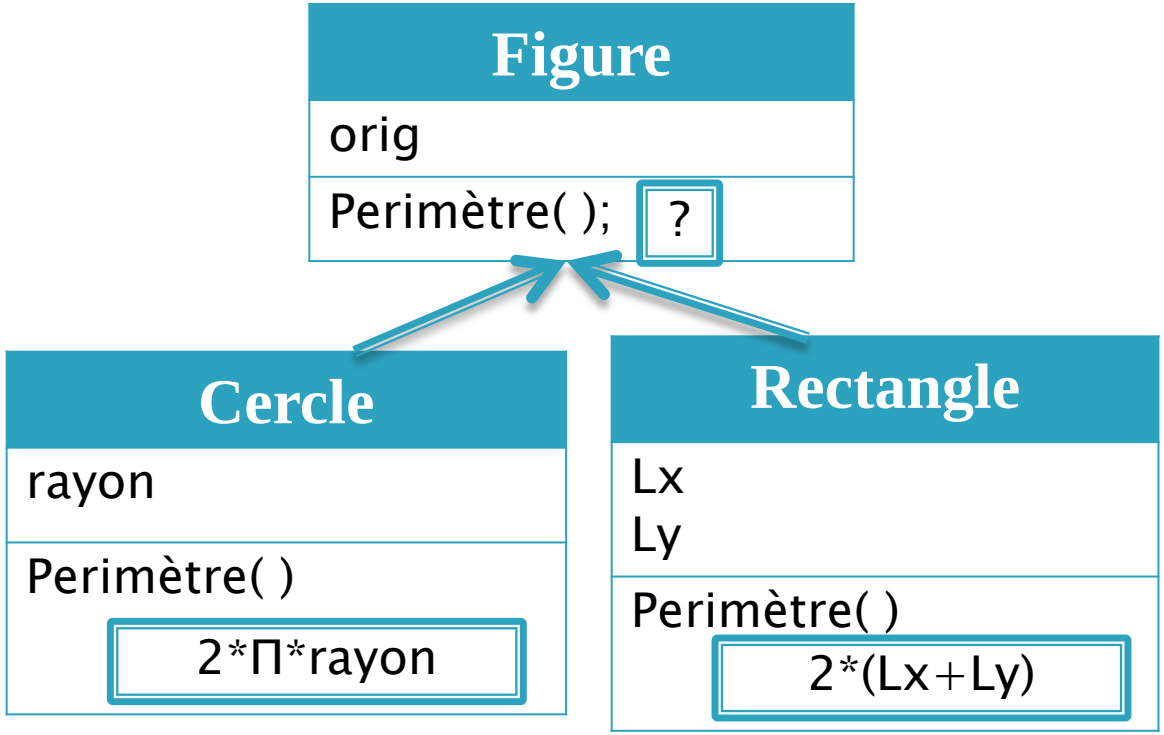
Dans cet exemple, la classe **A** annonce mais ne définit pas la méthode P**.** C'es**t** une classe abstraite. La classe dérivée **B** définit P et redéfinit Q.

Une classe abstraite n'ayant pas toutes ses méthodes définies, il est interdit de créer des objets de ce type.

```
A  a = new A( ) ; //interdit
```

Exemple

Comme exemple simple, on peut considérer diverses catégories de figures : les cercles, les rectangles etc..



Les diverses espèces de figures sont réalisées sous forme de classes héritières de **Figure** et chacune définit une version de la fonction **périmètre() :**

Exemple

```
abstract class Figure {
abstract double perimetre();
}
```

```
class Cercle extends Figure {
private static final double pi=3.141592;
double rayon;
Cercle(double r)
{ rayon=r;}
double perimetre() {return 2*pi*rayon;}
}
```

```
class Rectangle extends Figure
 {double LX, LY;
Rectangle(double lx,double ly)
{
 LX=lx; LY=ly;}
double perimetre() {return 2*(LX+LY);}
}
```

```
Forme f =new Circle(2);
Forme f =new Rectangle(2,3);
```

Les interfaces

Le langage *Java* ne permet pas de l'*héritage multiple* .
Pour supprimer les problèmes liés à l'héritage multiple, les *interfaces* sont des « sortes de classes» qui ne possèdent que des champs static final (autant dire des constantes) et des méthodes abstraites.

Une classe peut implémenter (**implements)** une ou plusieurs interfaces tout en héritant (**extends)** d'une classe.
Une interface peut hériter (**extends)** de plusieur (s) interface(s).

Les interfaces ne sont pas instanciables (Même raisonnement avec les classes abstraites)

```
NomInterface jeTente = new NomInterface(); // Erreur!!
```

Les interfaces

```
interface Service
{ int MAX = 1024 ;
int une_méthode(...) ;}
```

Dans la définition d'une interface seuls les droits d'accès public et le droit de paquetage (vide) sont autorisés.

```
package 1;

interface Operation
{ /*constantes*/
  public final double nombre =100 ;
            float x = 1000;
  /* que des methodes abstraites*/
   public double addition( );
   public float division( );
  double multiplication ( );
}
```

Exemple

Une contrainte dans l'implémentation d'une interface:
il faut obligatoirement redéfinir toutes les méthodes de l'interface

```
package 1;
abstract class Forme{  public abstract double perimetre( ) ;  }
interface Dessinable{public void dessiner();}
class Circle extends Forme  implement Dessinable
                {private double r;
                 public double perimetre ( ) { return 2 * Math.PI * r ; }
                 void dessiner ( ){ //instructions de dessin d un cercle}
                }
class Rectangle extends Forme implement Dessinable
               { private double long, larg;
                 public double perimetre( ) { return 2 * (long + larg); }
                    void dessiner ( ){ //instructions de dessin d un rectangle}
               }
     Dessinable dessins = new Circle (2); Dessinable dessins =new Rectangle(2,3),
```

CHAPITRE VI

POO: LES EXCEPTIONS

Les Exceptions

Définition

Une exception est un signal indiquant que quelque chose d'exceptionnelle(comme une erreur) s'est produite.

Elle interrompt le flot d'exécution normal du programme

Utilité:

Gérer les erreurs est indispensable : mauvaise gestion peut avoir des conséquences catastrophiques.

Vocabulaire

Lancer ou déclencher (**throw) une exception** consiste à signaler les erreurs.

Capturer ou attraper (**catch) une** exception permet de traiter les erreurs.

Les Exceptions

- Toutes les classes de Java possèdent des exceptions : par exemple, la classe **java.io**, qui gère les entrées - sorties, a, entre autres, l'exception **IOException etc** ...

Exemple de code générant une exception

```
public class CoursExc{
public static void main (String args[])
{
int j = 20, i = 0;
System.out.println(j/i);
System.out.println("coucou toi !");}}
```

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
at CoursExc.main(CoursExc.java:5)
)
```

```
public class CoursExc{
public static void main (String args[])
{int[] t=new int[3];
for(int j = 0;j<=3;j++)
{t[j]=j+1;
System.out.print
(t[j]);}
System.out.println("coucou toi !");}}
```

```
123Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsExceptio
n: 3
at CoursExc.main(CoursExc.java:5)
```

Capturer une exception

- Une exception se capture grâce à un bloc **try {……}**, lorsque le bloc try{...} capture une exception, le traitement associé se trouve dans un bloc **catch {……}.**
- **public class Exception {**
- **public static void main (String args[])**
- {
- **int j = 20, i = 0;**
- **try**{System.*out.println(j/i);}*
- **catch(ArithmeticException e)**
- {
- System.*out.println("Le denominateur ne peut être nul!");*
- }
- System.*out.println(« coucou c'est toi!");*
- }}

Capturer une exception

- **public class Exception {**
- **public static void main (String args[])**
- {
- **int j = 20, i = 0;**
- **try**
- {System.*out.println(j/i);*}
- **catch(ArithmeticException e)**
- {
- System.*out.println("Le denominateur ne peut être nul!"+e.getMessage());*
- }
- System.*out.println(« coucou c'est toi!");*
- }}

Lancer ou déclencher une exception

throws Une méthode déclare qu'elle peut lancer une exception par le mot clé **throws** Ce mot clé est suivi du nom de la classe qui va gérer l'exception.

throw Celui-ci permet d'instancier un objet dans la classe suivant l'instruction **throws**.

```
public class CompteBancaire {
 private double solde;
public void retrait(double montant) throws SoldeInsuffisant {
if( solde<montant) throw new SoldeInsuffisant
(" retrait de" + montant + "impossible");
solde -= montant;
}
 } public class SoldeInsuffisant extends Exception
{ SoldeInsuffisant(String msg) { super(msg); } }
```

TP sur les Exceptions(Lancer et Capturer une exception): Throws,throw

- Créer un classe personne avec une methode affiche() pour afficher les champs et deux constructeurs l'un sans argument et l'autre avec 3 arguments(Nom,Prenom,age). Mettre une condition dans le constructeur à 3 arguments qui si elle est remplie lance une exception de type AgeNegatifExc.

- Créer une classe TestPer qui permet de tester la classe personne.

Personne p=new Personne(« BA », «Bintou »,20)

Je me nomme BA Bintou

J'ai 20 ans

- Devoir de gérer les erreurs avec un bloc try{} catch{} sinon elles persistent.

TP sur les Exceptions(Lancer et Capturer une exception): Throws, throw, try{ }catch{ }

Capturer ou traiter l'exception au moment de l'instanciation dans la méthode main() et afficher les informations de la personne dans le bloc try.

Personne p=new Personne(« BA », »Bintou »,+/-20)

Deux possibilités:

Je me nomme: BA Bintou

J'ai 20 ans

Ou bien

Rien est exécuté

- Devoir de gérer les erreurs avec un bloc try{} catch{} sinon elles persistent.

TP sur les Exceptions(Lancer et Capturer une exception): Throws, throw, try{ }catch{ }

- Essayer d'afficher les informations à l'extérieur du bloc try{}.
- Que remarquez-vous?
- Essayer de résoudre le problème avec une variable globale.
- Que remarquer vous?
- Essayer de le résoudre avec le constructeur sans paramètres.
- Essayer de gérer deux exceptions à la fois (Exiger plus de 3 caractères pour le prénom par exemple.)
- Fin TP

Mme. KASSE Master I UCAD

Finally printStackTrace()

- la méthode **printStackTrace()** permet de voir ou se situe l'exception dans notre code ! Elle vous informe sur le nom de la classe levant l'exception et le numéro de ligne ou se trouve le code l'ayant levée.
- **Finally**: Quoi qu'il se passe dans la classe **try** ou **catch**, les instructions de la clause finally seront TOUJOURS exécutées.
- **Finally**
- {
- System.out.println(« ----------"); System.out.println("Voici le code qui est toujours exécuté ! ! ! ");
- System.out.println("----------\n");
- }

CHAPITRE VI

POO: LES ENTREES/SORTIES: (FLUX)

Présentation des FLux

- Les flux (stream en anglais) permettent d'encapsuler des processus d'envoi et de réception de données. Les flux traitent toujours les données de façon séquentielle.
- En java, les flux peuvent être divisés en plusieurs catégories :
  - les flux d'entrée (input stream) et les flux de sortie (output stream).
  - les flux de traitement de caractères et les flux de traitement d'octets.
- Java définit des flux pour lire ou écrire des données mais aussi des classes qui permettent de faire des traitements sur les données du flux. Ces classes doivent être associées à un flux de lecture ou d'écriture et sont considérées comme des filtres. Par exemple, il existe des filtres qui permettent de mettre les données traitées dans un tampon (buffer) pour les traiter par lots.

Toutes ces classes sont regroupées dans le package java.io.

Les Classes de gestion des FLux

- L'utilisation de ces classes n'est pas chose facile, vu leur nombre élevé et la difficulté de choisir la classe convenable à un besoin précis.
- Pour bien choisir une classe adaptée à un traitement donné, il faut comprendre la dénomination des différentes classes.
- Le nom d'une classe est toujours composé d'un **préfixe** et d'un **suffixe.**. Il y a quatre suffixes possibles en fonction du type de flux (flux d'octets ou de caractères) et du sens du flux (entrée ou sortie).

	Flux d'octets	Flux de caractères
Flux d'entrées	InputStream	Reader
Flux de sorties	OutputStream	Writer

Les Classes de gestion des FLux

Il existe donc quatre hiérarchies de classes qui encapsulent des flux particuliers.
Ces classes peuvent être séparées en deux séries de deux catégories différentes: les classes de lecture et d'écriture et les classes de manipulation de caractères et d'octets.

- ✓ les sous classes de **Reader** sont des types de flux en **lecture** sur les **caractères,**
- ✓ les sous classes de **Writer** sont des types de flux en **écriture** sur les **caractères,**
- ✓ les sous classes de **InputStream** sont des types de flux en **lecture** sur les **octets**,
- ✓ les sous classe de **OutputStream** sont des types de flux en **écriture** sur les **octets**.

Les Classes de gestion des FLux

On peut distinguer un ensemble de préfixes qui contiennent le type de traitement qu'il effectue.

Buffered : ce type de filtre permet de mettre les données du flux dans un tampon. Il peut être utilisé en entrée et en sortie.
Sequence : ce filtre permet de fusionner plusieurs flux.
Data : ce type de flux permet de traiter les octets sous forme de type de données.
LineNumber : ce filtre permet de numéroter les lignes contenues dans le flux.
PushBack : ce filtre permet de remettre des données lues dans le flux.
Print : ce filtre permet de réaliser des impressions formatées.
Object : ce filtre est utilisé par la sérialisation
InputStream / OuputStream : ce filtre permet de convertir des octets en caractères.

Buffered
Sequence
Data
LineNumber
PushBack
Print
Object
OutputStream

Les Flux de caractères en lecture/écriture

Ils transportent des données sous forme de caractères : java les gèrent avec le format Unicode qui code les caractères sur 2 octets. Les classes qui gèrent les flux de caractères héritent d'une des deux classes abstraites Reader ou Writer. Il existe de nombreuses sous classes pour traiter les flux de caractères.

	Flux en lecture	Flux en sortie
Flux de caractères	BufferedReader CharArrayReader FileReader InputStreamReader LineNumberReader PipedReader PushbackReader StringReader	BufferedWriter CharArrayWriter FileWriter OutputStreamWriter PipedWriter StringWriter

La classe Reader

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en lecture.
Cette classe définit plusieurs méthodes :

Méthodes	Rôles
boolean markSupported()	indique si le flux supporte la possibilité de marquer des positions
boolean ready()	indique si le flux est prêt à être lu
close()	ferme le flux et libère les ressources qui lui étaient associées
int read()	renvoie le caractère lu ou -1 si la fin du flux est atteinte.
int read(char[])	lire plusieurs caractères et les mettre dans un tableau de caractères
int read(char[], int, int)	lire plusieurs caractères. Elle attend en paramètre : un tableau de caractères qui contiendra les caractères lus, l'indice du premier élément du tableau qui recevra le premier caractère et le nombre de caractères à lire. Elle renvoie le nombre de caractères lus ou -1 si aucun caractère n'a été lu. Le tableau de caractères contient les caractères lus.

La classe FileReader

Cette classe permet de gérer les fichiers texte d'entrée. Elle dispose des mêmes fonctionnalités que la classe abstraite Reader.

Constructeur	Rôle
FileInputReader(String)	Créer un flux en lecture vers le fichier dont le nom est précisé en paramètre.
FileInputReader(File)	Idem mais le fichier est précisé avec un objet de type File

```
FileReader fichier = new FileReader("fichier.txt");
```

Il existe plusieurs méthodes de la classe FileReader qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe Reader et peuvent toutes lever l'exception IOException.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode **close().**

La classe Writer

C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de caractères en écriture.
Cette classe définit plusieurs méthodes :

Méthodes	Rôles
close()	ferme le flux et libère les ressources qui lui étaient associées
write(int)	écrire le caractère en paramètre dans le flux.
write(char[])	écrire le tableau de caractères en paramètre dans le flux.
write(char[], int, int)	écrire plusieurs caractères. Elle attend en paramètres : un tableau de caractères, l'indice du premier caractère dans le tableau à écrire et le nombre de caractères à écrire.
write(String)	écrire la chaîne de caractères en paramètre dans le flux
write(String, int, int)	écrire une portion d'une chaîne de caractères. Elle attend en paramètre : une chaîne de caractères, l'indice du premier caractère dans la chaîne à écrire et le nombre de caractères à écrire.

La classe FileWriter

Cette classe permet de gérer les fichiers texte d'entrée. Elle dispose des mêmes fonctionnalités que la classe abstraite Writer.

Constructeur	Rôle
FileWriter(String)	Si le nom du fichier précisé n'existe pas alors le fichier sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
FileWriter(File)	Idem mais le fichier est précisé avec un objet de la classe File.
FileWriter(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileWriter fichier = new FileWriter("fichier.txt");
```

Il existe plusieurs méthodes de la classe FileWriter qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe Writer et peuvent toutes lever l'exception IOException.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode **close().**

Les Flux de caractères tamponnés:

Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble de caractères représentant une ligne plutôt que de traiter les données caractères par caractères. Le nombre d'opérations est ainsi réduit.
Les classes **BufferedReader** et **BufferedWriter** permettent de gérer des flux de caractères tamponnés avec des fichiers.

La classe BufferedReader

Cette classe possède plusieurs constructeurs qui peuvent tous lever une exception de type FileNotFoundException:

Constructeur	Rôle
BufferedReader(Reader)	le paramètre fourni doit correspondre au flux à lire.
BufferedReader(Reader, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception de type IllegalArgumentException est levée.

```
BufferedReader fichier = new BufferedReader(new FileReader("fichier.txt"));
```

La classe BufferedReader

Il existe plusieurs méthodes de la classe BufferedReader héritées de la classe Reader qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes peuvent lever une exception de type IOException. Elle définit une méthode supplémentaire pour la lecture :

Méthode	Rôle
String readLine()	lire une ligne de caractères dans le flux. Une ligne est une suite de caractères qui se termine par un retour chariot '\r' ou un saut de ligne '\n' ou les deux.

La classe BufferedReader possède plusieurs méthodes pour gérer le flux hérité de la classe Reader.
Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple de lecture d'un fichier texte

```
import java.io.*; public class TestBufferedReader {
        protected String source;
        public TestBufferedReader(String source){
            this.source = source ;
            lecture();
        }

public static void main(String args[]) {
 TestBufferedReader =new TestBufferedReader("source.txt"); }

public void lecture() {
 try {
String ligne ;
   FileReader fr=new FileReader(source);
BufferedReader  br = new BufferedReader(fr);
while ((ligne = br.readLine()) != null)
{ System.out.println(ligne); }
 br.close();
fr.close(); }
 catch (Exception e)
 { e.printStackTrace(); }} }
```

La classe BufferedWriter

Cette classe possède plusieurs constructeurs Exemple:

Constructeur	Rôle
BufferedWriter(Writer)	le paramètre fourni doit correspondre au flux dans lequel les données sont écrites.
BufferedWriter(Writer, int)	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception IllegalArgumentException est levée.

```
BufferedWriter fichier = new BufferedWriter( new FileWriter("monfichier.txt"));
```

La classe BufferedWriter

Il existe plusieurs méthodes de la classe BufferedWriter héritées de la classe Writer qui permettent de lire un ou plusieurs caractères dans le flux.
La classe BufferedWriter possède plusieurs méthodes pour gérer le flux :

Méthode	Rôle
flush()	vide le tampon en écrivant les données dans le flux.
newLine()	écrire un séparateur de ligne dans le flux

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple d'écriture dans un fichier texte

```
import java.io.*; import java.util.*;
 public class TestBufferedWriter
 { protected String destination;

 public TestBufferedWriter(String destination)
 { this.destination = destination;
    traitement();
 }

private void traitement() {
try { String ligne ; int nombre = 123;
FileWriter fw=new FileWriter(destination);
BufferedWriter bw = new BufferedWriter(fw);
bw.write("bonjour tout le monde");
bw.newLine();
bw.write("Nous sommes le "+ new Date());
bw.write(", le nombre magique est " + nombre);
 fichier.close(); } catch (Exception e)
 { e.printStackTrace(); } } }
```

```
Public class TestBW
public static void main(String args[])
 {TestBufferedWriter tbw=new
TestBufferedWriter("print.txt"); }
```

La classe PrintWriter

Cette classe permet d'écrire dans un flux des données formatées.
Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
PrintWriter(Writer)	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.
PrintWriter(Writer, boolean)	Le booléen permet de préciser si le tampon doit être automatiquement vidé
PrintWriter(OutputStream)	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.
PrintWriter(OutputStream, boolean)	Le booléen permet de préciser si le tampon doit être automatiquement vidé

La classe PrintWriter

```
PrintWriter fichier = new PrintWriter( new FileWriter("monfichier.txt"));
```

Il existe de nombreuses méthodes de la classe PrintWriter qui permettent d'écrire un ou plusieurs caractères dans le flux en les formatant. Les méthodes write() sont héritées de la classe Writer. Elle définit plusieurs méthodes pour envoyer des données formatées dans le flux :

print(...)

Plusieurs méthodes print acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux

println()

Cette méthode permet de terminer la ligne courante dans le flux en y écrivant un saut de ligne.

println (...)

Plusieurs méthodes println acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux avec une fin de ligne.

La classe PrintWriter possède des méthodes pour gérer le flux :

Méthode	Rôle
flush()	Vide le tampon en écrivant les données dans le flux.

Exemples

```
import java.io.*;
 import java.util.*;
public class TestPrintWriter {
protected String destination;
public TestPrintWriter(String destination) {
this.destination = destination;
traitement(); }
public static void main(String args[]) {
new TestPrintWriter("print.txt"); }
private void traitement() {
try { String ligne ; int nombre = 123;
PrintWriter fichier = new PrintWriter(new FileWriter(destination));
fichier.println("bonjour tout le monde");
fichier.println("Nous sommes le "+ new Date());
fichier.println("le nombre magique est " + nombre);
fichier.close(); } catch (Exception e)
{ e.printStackTrace(); } } }
```

Les Flux d'octets

Ils transportent des données sous forme d'octets. Les flux de ce type sont capables de traiter toutes les données.
Les classes qui gèrent les flux d'octets héritent d'une des deux classes abstraites InputStream ou OutputStream. Il existe de nombreuses sous classes pour traiter les flux d'octets.

Les flux d'octets avec un fichier.

Les classes FileInputStream et FileOutputStream permettent de gérer des flux d'octets avec des fichiers.

Les flux d'octets en lecture sur un fichier

Il faut instancier un objet de la classe FileInputStream. Cette classe possède plusieurs constructeurs qui peuvent tous lever l'exception FileNotFoundException:

Constructeur	Rôle
FileInputStream(String)	Ouvre un flux en lecture sur le fichier dont le nom est donné en paramètre
FileInputStream(File)	Idem mais le fichier est précisé avec un objet de type File

```
FileInputStream fichier = new FileInputStream("monfichier.dat");
```

Les flux d'octets en lecture sur un fichier

Il existe des méthodes de la classe FileInputStream qui permettent de lire un ou plusieurs octets dans le flux. Toutes ces méthodes peuvent lever l'exception IOException.

int read( )

Cette méthode envoie la valeur de l'octet lu ou -1 si la fin du flux est atteinte.

int read(byte[], int, int), int read(byte[ ])

Cette méthode lit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contiendra les octets lus, l'indice du premier élément du tableau qui recevra le premier octet et le nombre d'octets à lire.

Elle renvoie le nombre d'octets lus ou -1 si aucun octet n'a été lu. Le tableau d'octets contient les octets lus.

Les flux d'octets en lecture sur un fichier

La classe FileInputStream possède plusieurs méthodes pour gérer le flux :

Méthode	Rôle
long skip(long)	saute autant d'octets dans le flux que la valeur fournie en paramètre. Elle renvoie le nombre d'octets sautés.
close()	ferme le flux et libère les ressources qui lui étaient associées
int available()	retourne une estimation du nombre d'octets qu'il est encore possible de lire dans le flux

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Les flux d'octets en écriture sur un fichier

Il faut instancier un objet de la classe FileOutputStream. Cette classe possède plusieurs constructeurs :

Constructeur	Rôle
FileOutputStream(String)	Si le fichier précisé n'existe pas, il sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
FileOutputStream(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileOuputStream fichier = new FileOutputStream("monfichier.dat");
```

Les flux d'octets en écriture sur un fichier

Il existe plusieurs méthodes de la classe FileOutputStream qui permettent de lire un ou plusieurs octets dans le flux.

write(int)

Cette méthode écrit l'octet en paramètre dans le flux.

write(byte[])

Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire : tous les éléments du tableau sont écrits.

write(byte[], int, int)

Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire, l'indice du premier élément du tableau d'octets à écrire et le nombre d'octets à écrire.

Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode close().

Exemple

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class CopieFichier {
public static void main(String args[]) {
try { copierFichier("source.txt", "copie.txt"); }
catch (IOException e) { e.printStackTrace(); } }
 public static void copierFichier(String source, String destination)
throws IOException {
FileInputStream fis = null;
FileOutputStream fos = null; try { byte buffer[] = new byte[1024];
int taille = 0;
fis = new FileInputStream(source);
fos = new FileOutputStream(destination);
while ((taille = fis.read(buffer)) != -1)
{ System.out.println(taille);
fos.write(buffer, 0, taille); } }
 finally { if (fis != null) { try { fis.close(); } catch (IOException e) { }
} if (fos != null) { try { fos.close(); } catch (IOException e) { } } } } }
```

La classe File

Une instance de la classe File est une représentation logique d'un fichier ou d'un répertoire qui peut ne pas exister physiquement sur le disque.
Si le fichier ou le répertoire existe, de nombreuses méthodes de la classe File permettent d'obtenir des informations sur le fichier. Sinon plusieurs méthodes permettent de créer des fichiers ou des répertoires. Voici une liste de quelques méthodes :

Méthode	Rôle
boolean canRead()	indique si le fichier peut être lu
boolean canWrite()	indique si le fichier peut être modifié
boolean exists()	indique si le fichier existe physiquement
String getAbsolutePath()	renvoie le chemin absolu du fichier
String getPath	renvoie le chemin du fichier
boolean isDirectory()	indique si le fichier est un répertoire
long length()	renvoie la longueur du fichier
String[] list()	renvoie la liste des fichiers et répertoire contenu dans le rép
boolean mkdir()	création du répertoire
boolean mkdirs()	création du répertoire avec création des répertoires manquants dans l'arborescence du chemin

Utilisation de quelques méthodes

```
import java.io.*;
 public class TestFile {
 protected String nomFichier;
protected File fichier;
 public TestFile(String nomFichier) { this.nomFichier = nomFichier;
 fichier = new File(nomFichier);
 traitement(); }
private void traitement() {
 if (!fichier.exists()) {
 System.out.println("le fichier "+nomFichier+"n'existe pas");
 System.exit(1); }
 System.out.println(" Nom du fichier : "+fichier.getName());
 System.out.println(" Chemin du fichier : "+fichier.getPath());
 System.out.println(" Chemin absolu : "+fichier.getAbsolutePath());
 System.out.println(" Droit de lecture : "+fichier.canRead());
 System.out.println(" Droit d'ecriture : "+fichier.canWrite());
 if (fichier.isDirectory())
 { System.out.println(" contenu du repertoire ");
String fichiers[] = fichier.list();
for(int i = 0; i < fichiers.length; i++)
System.out.println(" "+fichiers[i]); } } }
```

```
 public class TestMethodesFile
public static void main(String args[]) {
 new TestFile(args[0]); }
```

Les fichiers à accès direct

Les fichiers à accès direct permettent un accès rapide à un enregistrement contenu dans un fichier.

La classe RamdonAccessFile encapsule les opérations de lecture/écriture d'un tel fichier. Elle implémente les interfaces **DataInput** et **DataOutput.**
Elle possède deux constructeurs qui attendent en paramètre **le fichier à utiliser** (sous la forme d'un nom de fichier ou d'un objet de type File qui encapsule le fichier) et **le mode d'accès**.
Le mode est une chaîne de caractères qui doit être égal à **«r»** ou **«rw»** selon que le mode est lecture seule ou lecture/écriture.
Ces deux constructeurs peuvent lever les exceptions suivantes :
FileNotFoundException si le fichier n'est pas trouvé
IllegalArgumentException si le mode n'est pas «r» ou «rw»
SecurityException si le gestionnaire de sécurité empêche l'accès aux fichiers dans le mode précisé

Les fichiers à accès direct

La classe RandomAccessFile possède de nombreuses méthodes writeXXX() pour écrire des types primitifs dans le fichier

```
package 1
import java.io.RandomAccessFile;
public class TestRandomAccesFile {
public static void main(String[] args) {
try {
RandomAccessFile monFichier = new RandomAccessFile("monfichier.txt", "rw"); for
(int i = 0; i < 10; i++)
{ monFichier.writeInt(i * 100); }
monFichier.close(); }
catch (Exception e)
{ e.printStackTrace(); } } }
```

Les fichiers à accès direct

Elle possède aussi de nombreuses classes readXXX() pour lire des données primitives dans le fichier.

```
package 1
import java.io.RandomAccessFile;
public class TestRandomAccesFile {
public static void main(String[] args) {
try {
RandomAccessFile monFichier = new RandomAccessFile("monfichier.txt", "rw");
for (int i = 0; i < 10; i++)
{ System.out.println(monFichier.readInt()); }
monFichier.close(); }
catch (Exception e)
{ e.printStackTrace(); } } }
```

CHAPITRE VII

GUI: LES INTERFACES GRAPHIQUES

LES SWING GUI

- Les classes graphiques Swing dérivent de la classe **JComponent** , qui hérite elle-même de la classe AWT (Abstract Window Toolkit) **Container**.
  - Composants SWING VS COMPOSANT AWT
- Tous les composants du package AWT sont des composants lourds.
- Les composants Swing (nom du package : javax.swing) sont pour la majorité d'entre eux des composants légers.
- Tous les composants Swing commencent par la lettre " J ".
- Les composants Swing se répartissent :
  - en conteneurs de plus lourd:
    - **JFrame** à rapprocher de la classe **Frame** dans AWT
    - **JDialog** à rapprocher de la classe **Dialog** dans AWT
    - **JWindow** à rapprocher de la classe **Window** dans AWT
    - **JApplet** à rapprocher de la classe **Applet** dans AWT
  - en conteneurs légers (les autres contrôles GUI Swing).

LES SWING GUI

```
java.lang.Object
+--java.awt.Component
+--java.awt.Containe

+--java.awt.Window
| +--javax.swing.JWindow

| +--java.awt.Frame
| | +--javax.swing.JFrame

| +--java.awt.Dialog
| +--javax.swing.JDialog

+--java.awt.Panel
|
+--java.applet.Applet
+--javax.swing.JApplet
```

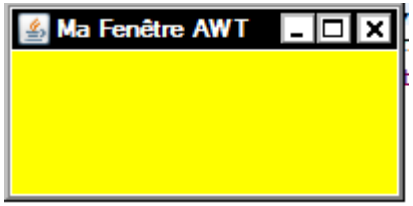
Pour les Swing, un conteneur de plus haut niveau se compose d'une " fenêtre visible " , la ContentPane

Attention : avec AWT le dépôt du composant s'effectue directement sur le conteneur. Il faut en outre éviter de mélanger des AWT et des Swing sur le même conteneur.

LES SWING GUI

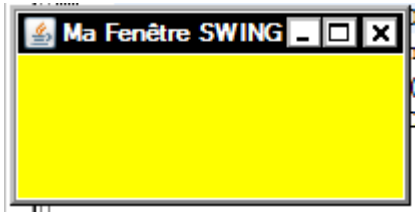
AWT

```
import java.awt.*;
public class Fen_Awt {
public static void main(String args[])
{
Frame fen=new Frame("Ma fenêtre AWT");
fen.setSize(100,100);
    fen.setBackground(Color.yellow);
    fen.setVisible(true);
}
}
```



SWING

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing {
 public static void main(String args[]){
JFrame fen=new JFrame("Ma Fenêtre SWING");
Container ct=fen.getContentPane();
fen.setSize(200,100);
ct.setBackground(Color.yellow);
fen.setVisible(true);
 }
}
```



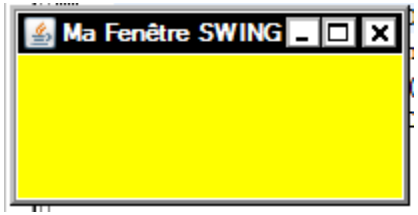
Création d'une fenêtre personnalisée

- Nous allons créer notre propre classe dérivée de JFrame et créer un objet de ce type.

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing extends JFrame {
public Fen_Swing extends
Container ct=getContentPane();
this.setSize(100,100);
This.setTitle ("Ma Fenêtre SWING ");
ct.setBackground(Color.yellow);
 }
```

```
public class Fenêtre{
 public static void main(String
args[]){
Fen_Swing fs= new Fen_Swing();
Fs.setVisible(true);

}
}
```



Action sur les caractéristiques d'une fenêtre

- La methode **setBounds** permet non seulement de fixer les dimensions mais aussi la position de la fenêtre à l'ecran.
- Exemple:
- **setBounds(50,50,300,200);**

  le coin supérieur gauche de la fenêtre est placé au pixel de coordonnées 50,50 et ses dimensions seront de 300*200 pixels.
- **setBackground(Color.c)** // Modification de la couleur de fond.
- On peut imposer un autre comportement de la fenêtre en appelant la méthode **setDefaultCloseOperation** avec l'un des arguments suivants**:**
  - **DO_NOTHING_ON_CLOSE** //ne rien faire
  - **HIDE_ON_CLOSE** //cacher la fenêtre(comportement par défaut)
  - **DISPOSE_ON_CLOSE** //détruire l'objet fenêtre

Gestion de clic dans une fenêtre

1. **Implémentation de l'interface MouseListener**
2. Il existe un catégorie d'événements souris qu'on peut traiter avec un ecoteur de souris ie un objet d'une classe implémentant l'interface **MouseListener.** Cette dernière comporte Cinq methodes correspondant chacune à un événement particulier:

- **mousePressed, mouseReleased, mouseEntered, mouseExited, mousecliClicked**

La methode **addMouseListener** figure dans toutes les classes sucsceptibe de générer des événements souris: **addMouseListener(objetEcouteur)**

La gestion des événements fait appel au paquetage **java.awt.event.**

```
Public class EcoSoris implements MouseListener{
   public void mouseClicked( MouseEvent e)    { }
   public void mouseReleased( MouseEvent e)  { }
   public void mouseExited( MouseEvent e)      { }
   public void mousePressed( MouseEvent e)    { }
   public void mouseEntered( MouseEvent e)    { }
}
```

EXEMPLE

```
import java.awt .*;
import java.awt.event.*;
import javax.swing.*;
public class Fenêtre extends JFrame  implements MouseListener
{
    public Fenêtre() {
    this.setTitle(" Gestion de clic");
    this.setBounds(100,100,250,200);
                    Container contenu = this.getContentPane();
                     contenu.setBackground(Color.yellow);
   this. addMouseListener(this);
}
public void mouseClicked(Mouseevent ev)
{System.out.println("J'ai cliquer dan la fenêtre");}
   public void mouseReleased( MouseEvent e)   {  }
   public void mouseExited( MouseEvent e)    {  }
   public void mousePressed( MouseEvent e)  {  }
   public void mouseEntered( MouseEvent e)  {  }
}
```

```
public class Clic{
 public static void
main(String args[]){
Fenêtre f= new
Fenêtre();
Fs.setVisible(true);
}
}
```

Gestion de clic dans une fenêtre

- L'argument transmis à la méthode **mouseClicked** est un objet de type **MouseEvent** qui est une classe correspondant à la catégorie d'événements gérés par l'interface **MouseListener**. Elle contient l'information sur les coordonnées du curseur de souris au moment du clic grâce à ces méthodes **getX** et **getY.**
- **Exemple.**

```
public class Fenêtre extends JFrame  implements
MouseListener {
    public Fenêtre() {
    this.setTitle(" Gestion de clic");
    this.setBounds(100,100,250,200);
    Container contenu = this.getContentPane();
contenu.setBackground(Color.yellow);
   this. addMouseListener(this);
}
public void mousePressed ( MouseEvent ev)
  {System.out.println ("vous avez appuyé au point de
coordonnes : "+ev.getX()+" "+ev.getY());
  }
   public void mouseReleased ( MouseEvent ev)
 {System.out.println ("vous avez appuyé au point de
coordonnes : "+ev.getX()+" "+ev.getY());}
```

```
public void mouseExited ( MouseEvent e)     {  }
public void mouseClicked ( MouseEvent e)    {  }
public void mouseEntered ( MouseEvent e)    {  }
}
```

```
public class Clic{
 public static void
main(String args[]){
Fenêtre fs= new
Fenêtre();
fs.setVisible(true);
}}
```

La notion d'adaptateur

- Java dispose d'une classe particulière appelée **MouseAdapter** qui implémente toutes les méthodes de l'interface **MouseListener** avec un corps vide. Tout se passe comme ci **MouseAdaptater** était définie ainsi:

- **class** MouseAdapter **implements** MouseListener
- { public void mouseReleased (MouseEvent e) { }
- public void mouseExited (MouseEvent e) { }
- public void mousePressed (MouseEvent e) { }
- public void mouseEntered (MouseEvent e) { }
- public void mouseClicked (MouseEvent e) { }
- }

La notion d'adaptateur

```
import java.awt .*;    import java.awt.event.*;    import javax.swing.*;
public class Adapt extends JFrame   {
    public Adapt (String titre) {
    this.setTitle(titre);
    this.setBounds(100,100,250,200);
    Container contenu = this.getContentPane( );
    contenu.setBackground(Color.yellow);
/* on cree un objet ecouteur de la fenetre*/

EcouteurFenetre ecout=new EcouteurFenetre();

this.addMouseListener(ecout);
 }
}
```

```
import java.awt.event.*;
class EcouteurFenetre extends MouseAdapter{

public void mouseClicked( MouseEvent ev)
  {
   System.out.println ("vous avez clique au point de coordonnes : "+ev.getX()+" "+ev.getY());
  }
}
```

La notion d'adaptateur(Utilisation de classe Anonyme)

```
import java.awt .*;    import java.awt.event.*;    import javax.swing.*;
public class Adapt extends JFrame   {
    public Adapt (String titre) {
    this.setTitle(titre);
    this.setBounds(100,100,250,200);
    Container contenu = this.getContentPane( );
    contenu.setBackground(Color.yellow);
/* on cree un objet ecouteur de la fenetre*/
this.addMouseListener(new MouseAdapter()
{ public void MouseClicked(MouseEvent ev){ System.out.Println(Vous avez cliqué sur le point de cooedonnées +ev.getX() +ev.getY());}});
 }
}
```

Ajout d'un bouton à une fenêtre

- **JButton** UnBouton = **new JButton(**" bouton1"**);** *// instanciation d'un JButton*
- **Container** c = LaFenetre.**getContentPane**( ) ; *// obtention de la référence du contentPane du JFrame*
- c.add(UnBouton) ; *// on dépose le JButton sur le JFrame à travers son ContentPane*

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing extends JFrame {
Public Fen_Swing(String titre)
Container ct=getContentPane();
this.setBounds(100,100,250,300);
This.setTitle (titre);
ct.setBackground(Color.yellow);
JButton bout1 = new JButton ("Essai");
 bout1.setBackground (Color.red);
 ct.add (bout1);
 }
```

Mise en forme d'un bouton

- Java utilise un gestionnaire de classe BorderLayout avec lequel, en l'absence d'informations spécifiques, un composant occupe toute la fenêtre.
- BorderLayout place les composants au quatre points
- Cardinaux (North, West, East, South) et au centre (Center).

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing extends JFrame {
Public Fen_Swing(String titre)
Container ct=getContentPane();
this.setBounds(100,100,250,300);
This.setTitle (titre);
ct.setBackground(Color.yellow);
JButton bout1 = new JButton("Valider");
 bout1.setBackground (Color.red);
JButton bout2 = new JButton(« Annuler");
bout1.setBackground (Color.green);
ct.add (bout1, BorderLayout.NORTH);
 ct.add (bout2, BorderLayout.SOUTH);
 }
```

Mise en forme d'un bouton

- Il existe un gestionnaire plus intéressant de la **classe FlowLayout** qui dispose les différents composants en « flots » ie qu'il les affiche un peu comme du texte , les un à la suite des autres, d'abord sur une même ligne ensuite line après ligne.

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing extends JFrame {
Public Fen_Swing(String titre)
Container ct=getContentPane();
this.setBounds(100,100,250,300);
This.setTitle (titre);
ct.setBackground(Color.yellow);
JButton bout1 = new JButton("Valider");
JButton bout2 = new JButton(« Annuler");
 bout1.setBackground (Color.red);
bout2.setBackground (Color.green);
ct.add (bout1);
 ct.add (bout2;
ct.setLayout(new FlowLayout()); }
```

Gestion d'un bouton avec un écouteur

- Un bouton gère une catégorie d'évènement appelée **Action** qu'on l'on traite avec un écouteur qui est un objet implémentant l'interface ActionListener.
- Cette dernière ne possède qu'une seule méthode :
- public void actionPerformed (ActionEvent ev).
- L'ecouteur est associé au bouton par la methode addActionListener();

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing extends JFrame
implements ActionListener{
Public Fen_Swing(String titre)
Container ct=getContentPane();
this.setBounds(100,100,250,300);
This.setTitle (titre);
ct.setBackground(Color.yellow);
JButton bout1 = new JButton("Valider");
JButton bout2 = new JButton(« Annuler");
bout1.setBackground (Color.red);
bout2.setBackground (Color.red);
ct.add (bout1);
ct.add (bout2);
ct.setLayout(new FlowLayout());
bout1.addActionListener(this);
bout2.addActionListener(this); }
Public void actionPerformed(ActionEvent av)
{ System.out.println("Action sur bouton");}}}
```

Gestion de plusieurs composants

- La méthode **gerSource** (présente dans toutes les classes événements, donc dans **ActionEvent**) fournit une référence (de type Object) sur l'objet ayant déclenché l'événement concerné.

```
Public void actionPerformed(ActionEvent av)
{
   if(ev.getSource()==bout1)
    System.out.println("Action sur Valider");
 if(ev.getSource()==bout2)
    System.out.println("Action sur Annuler");


}
```

Gestion de plusieurs composants

- La méthode getSource permet d'identifier la source d'un événement et elle a le mérite de s'appliquer à tous les événements générés par tous les composants.
- La méthode getActionCommand, présente uniquement dans la classe ActionEvent, permet d'obtenir la chaine de commande associée à la source d'un événement.

```
Public void actionPerformed(ActionEvent av)
{String nom=ev.getActionCommand();
System.out.println("Action sur "+ nom);
}
```

Création de panneau

- Un panneau est une sorte de sous-fenêtre », sans titre ni bordure. Il s'agit donc d'un simple rectangle qui, quand on ne lui donne pas de couleur spécifique n'est guère visible.

```
import java.awt .*;    import java.awt.event.*;    import javax.swing.*;
public class Panneau extends JFrame   {
JPanel pan;
    public Panneau (String titre) {
    this.setTitle(titre);
    this.setBounds(100,100,250,200);
    Container contenu = this.getContentPane( );
pan=new JPanel();
pan.setBackground(Color.yellow);
Contenu.add(pan)
}
}
```

Création de panneau

- Pour dessiner dans un panneau, il faut *redéfinir* la méthode **paintComponent**
- (appartenant à la classe **JPanel**).
- **Void paintComponent(Graphic g)**
- La méthode **drawLine** dessine un trait dans un panneau
- **g.drawLine(15,10,40,50);**
- Il faut aussi appeler la méthode **paintComponent** de la classe ascendante JPanel avec:
- **Super.paintComponent(g);**

```
public class Pan extends JPanel{
public void paintComponent(Graphics g){
super.paintComponent(g);
g.drawLine(15,10,100,100);
}
}
```

Les Cases à cocher

- Les cases à cocher permettent à l'utilisateur d'effectuer un choix de type oui/non Elle est instancier grâce à la classe JCheckBox.
- JCheckBox jch=new JCheckBox(" marié(e) ");
- Ct.add(jch);

```
import javax.swing.*;
import java.awt.*;
public class Fen_Swing extends JFrame {
Public Fen_Swing(String titre){
container ct=getContentPane();
this.setBounds(100,100,250,300);
this.setTitle (titre);
ct.setBackground(Color.yellow);
JButton bout1 = new JButton("Valider");
 bout1.setBackground (Color.red);
JButton bout2 = new JButton(« Annuler");
bout1.setBackground (Color.green);
ct.add (bout1); ct.add (bout2);
JCheckBox jch1=new JCheckBox(" Marié(e) ");
Ct.add(jch1);
JCheckBox jch2=new JCheckBox(" Célibataire");
Ct.add(jch2);
Ct.setLayout(new FlowLayout());
 }
```

Action sur une cases à cocher

- Une action sur une case à cocher provoque:
  - un événement de type Action
  - Un événement de type Item

La méthode **isSelected** permet de savoir l'état d'une case à cocher

La methode **setSlected(booleen b)** permet de changer l'état d'une case à cocher selon la valeur de b.

```
public void actionPerformed(ActionEvent av)
{  String nom=av.getActionCommand();
      if(av.getSource()==jch1&&jch1.isSelected())
      System.out.println("la case à cocher" +nom+" est selectionnée");
      if(av.getSource()==jch2&&jch2.isSelected())
      System.out.println("la case à cocher" +nom+" est selectionnée");
}
```

Action sur une cases à cocher

Un écouteur de catégorie Item doit implémenter l'interface **ItemListener** qui comporte une seule méthode **ItemStateChanged.**

Public void ItemStateChanged(ItemEvent iv)

```
public void itemStateChanged(ItemEvent
iv){if(iv.getSource()==jch1&&jch1.isSelected())
System.out.println("la case à cocher marié est selectionnée");
if(iv.getSource()==jch2&&jch2.isSelected())
System.out.println("la case à cocher célibataire est
selectionnée");
}
```

Les boutons radio

- Les boutons radio permettent d'exprimer un choix de type oui/non mais le choix d'une option du groupe entraine automatiquement la désactivation de l'option choisie précédemment. La classe JRadioButton permet d'instancier un tel objet.
- **JRadioButton jr1=new JRadioButton ("Vert");**
- **JRadioButton jr2=new JRadioButton("Blanc")**

- Pour obtenir la désactivation automatique d'autre boutons radio d'un même
- groupe, il faut créer un objet de type **ButtonGroup:**
- **ButtonGroup bg=new ButtonGroup();**

- Associer chaqu'un des boutons au groupe avec la méthode **add()**
- **bg.add(jr1);**
- **bg.add(jr2);**

Action sur les boutons radio

- **Même raisonnement que les cases à cocher.**

```
public void actionPerformed(ActionEvent av)
{  String nom=av.getActionCommand();
if(av.getSource()==jr1&&jr1.isSelected())
System.out.println("le bouton radio" +nom+" est selectionné");
if(av.getSource()==jr2&&jr2.isSelected())
System.out.println("le bouton radio" +nom+" est selectionné");
}
```

Les étiquettes et les champs texte

- Pour afficher du texte dans un conteneur on utilise la classe **JLabel:**
- **JLabel jl=new JLabel("Factoriel");**
- Le texte d'une étiquette peut être modifier par la méthode **setText()**
- Un champ de texte est une zone rectangulaire dans laquelle l'tilisateur peut entrer ou modifier des valeurs.
- On utilise la classe **JTextField** pour instancier un tel objet.
- **JTextField jt1,jt2;**
- **Jt1=new JTextField(15);**
- **Jt2=new JTetField("texte par défaut",15);**
- L'information figurant dans un champs texte s'obtient avec la méthode **getText()**

Les boites de listes

- Les boites de liste sont des composants qui permettent de choisir une ou plusieurs valeurs parmi une liste de valeurs prédéfinies.
- On crée un tel objet en fournissant un tableau de chaines à son constructeurs **JList()**
- String[] jour={**"Lundi", "Mardi ", "Mercredi ", "Jeudi ", "Vendredi", " Samedi ", "Dimanche"}**
- **JList jls=new JList(jour);**
- On sélectionne un élément de rang donné par la méthode **setSelectedIndex().**
- **jls.setSelectedIndex(2).**
- Pour utiliser la barre de défilement on introduit la liste dans un panneau de type **JScrollPane.**
- **JScrollPane defil =new JScrollPane(jls);**
- **ct. add(defil);**
- **jls.setVisibleRowCount(3);**
- La méthode getSelectedValue() permet d'accéder à l'information sélectionnée
- **String ch=(String)jls.getSelectedValue();**
- Pour selectionner plusieurs valeurs on utilise la méthode getSelectedValues() qui fournit un tableau d'Object;
- **Object[] Ob=jls.getSlectedValues();**
- **for(int i=0;i<ob.length;i++)**
- **System.out.println((String)ob[i]);**

Evénement générés par les boites de listes

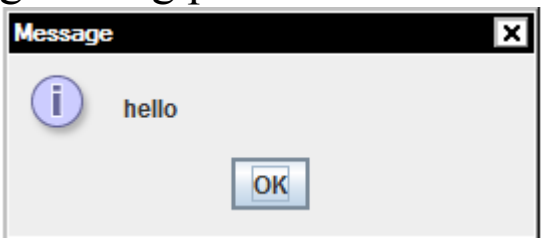
- Les boites de liste ne génèrent pas d'évènement action. On les traite par un écouteur d'une classe implémentant l'interface ListSelectionListener qui comporte la méthode valuechanged().
- Public void valueChanged (LisSelectionEvent ls)
- La méthode getValueIsAdjusting() permet de savoir si l'on est ou non en phase de transition.
- public void valueChanged (LisSelectionEvent ls){
- If(!ls.getValueIsAdjusting(){//traitement}
- }
- **public void** valueChanged (ListSelectionEvent ls){
- **if**(!ls.getValueIsAdjusting()){
- {
- System.*out*.println("Les jours selectionnés sont: ");
- Object[] ob=jls.getSelectedValues();
- **for(int** i=0;i<ob.length;i++)
- System.*out*.println((String)ob[i]);
- }}

Les boites Combo

- Les boites combo associent un champ de texte et une boite de liste à sélection multiple. On utilise la classe JComboBox
- String[] mois={**"Janvier", "Février", "Mars", "Avril", "Mai", "Juin", "Juillet"}**
- JComboBox jco=new JComboBox(mois);
- La boite combo génère des évènements action.
- **public void** actionPerformed(ActionEvent av)
- { **if**(av.getSource()==jco){
- Object obj=jco.getSelectedItem();
- System.*out*.println("Il est né au moi de: "+(String)obj);
- }

Les boites de dialogue(Les boites de Message)

- La classe JOptionPane avec sa méthode static showMessageDialog permettent d'afficher et de gérer automatiquement les boites de dialogue.



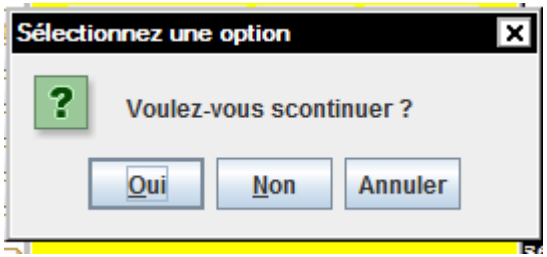
- System.*out*.println("Avant");
- JOptionPane.*showMessageDialog*(**null**,"hello");
- System.*out*.println("Après");
- Il existe une autre variante de la méthode *showMessageDialog() qui permet de choisir : le contenu, le titre de la boite, le type d'icône.*

Paramètre	**Type d'icône**
JOptionPane.ERROR_MESSAGE	Erreur
JOptionPane.INFORMATION MESSAGE	Information
JOptionPane.WARNING_MESSAGE	Avertissement
JOptionPane.QUESTION_MESSAGE	Question
JOptionPane.PLAIN_MESSAGE	Aucune icône

Exemple: *showMessageDialog(fs,* "Erreur ", " Message d'erreur ", JOptionPane.ERROR_MESSAGE...)

Les boites de dialogue(Les boites de confirmation)

Pour les boites de confirmation java utilise la méthode showConfirmDialog() de La classe JOptionPane.



Oui→0

Non→1

Annuler→2

X→-1

```
int rep=JOptionPane.showConfirmDialog(fs,
"Voulez-vous continuer ?");
 if(rep==0)
 System.out.println("Vous avez accepté de
continuer");
 else if(rep==1)
 System.exit(0);
 else if(rep==2)
 System.out.println("Vous venez d'annuler");
 else
 System.out.println("Fin");
```

Les boites de saisie

- La méthode showInputDialog de la classe JOptionPane permet de gérer automatiquement le dialogue avec l'utilisateur.
- JOptionPane.showInputDialog(fen, "Donner un text").
- Cette méthode fournit en retour:
- Soit un String contenant le texte fourni par l'utilisateur
- Soit la valeur null si l'utilisateur n'a pas confirmé sa saisie avec OK.
- String s= JOptionPane.*showInputDialog*(fs,"Donner votre faculte","ETUDIANT",JOptionPane.*WARNING_MESSAGE*);
- **if**(s.length()==0)
- System.*out*.println("Vous n'êtes pas étudiant");
- **else**
- System.*out*.println("Vous êtes étudiant à la faculté de: "+s);

Les Menus

- Le menus déroulants usuels font intervenir trois sortes d'objets:
  - Un objet barre de menu**(JMenuBar)**
  - Différents objets menu**(JMenu)** qui seront visibles dans la barre de menus
  - Pour chaque menu, les différentes options, de type **JMenuItem**, qui le constituent.

```
JMenuBar jmb=newJMenuBar();
fs.setJMenuBar(jm);
JMenu jm1=new JMenu("Fichier ");
JMenu jm2=new JMenu("Dessin");
JMenu jm3=new JMenu("Couleur");
jmb.add(jm1);
jmb.add(jm2);
JMenuItem jmi1=new JMenuItem("Ouvrir"); JMenuItem v=new
   JMenuItem("Verte");
JMenuItem jmi2=new JMenuItem("Fermer"); JMenuItem r=new
   JMenuItem("Rouge");
JMenuItem j=new JMenuItem("Jaune");
jm1.add(jmi1); jm1.add(jmi2); jm2.add(jm3); jm3.add(v); jm3.add(j); jm3.add(r);
```

  - if(av.getSource()==r){this.ct.setBackground(Color.*red*);}
  - if(av.getSource()==jmi1){Swing05 sg5=new Swing05("Calculatrice"); sg5.setVisible(true);}

CHAPITRE VIII

JDBC

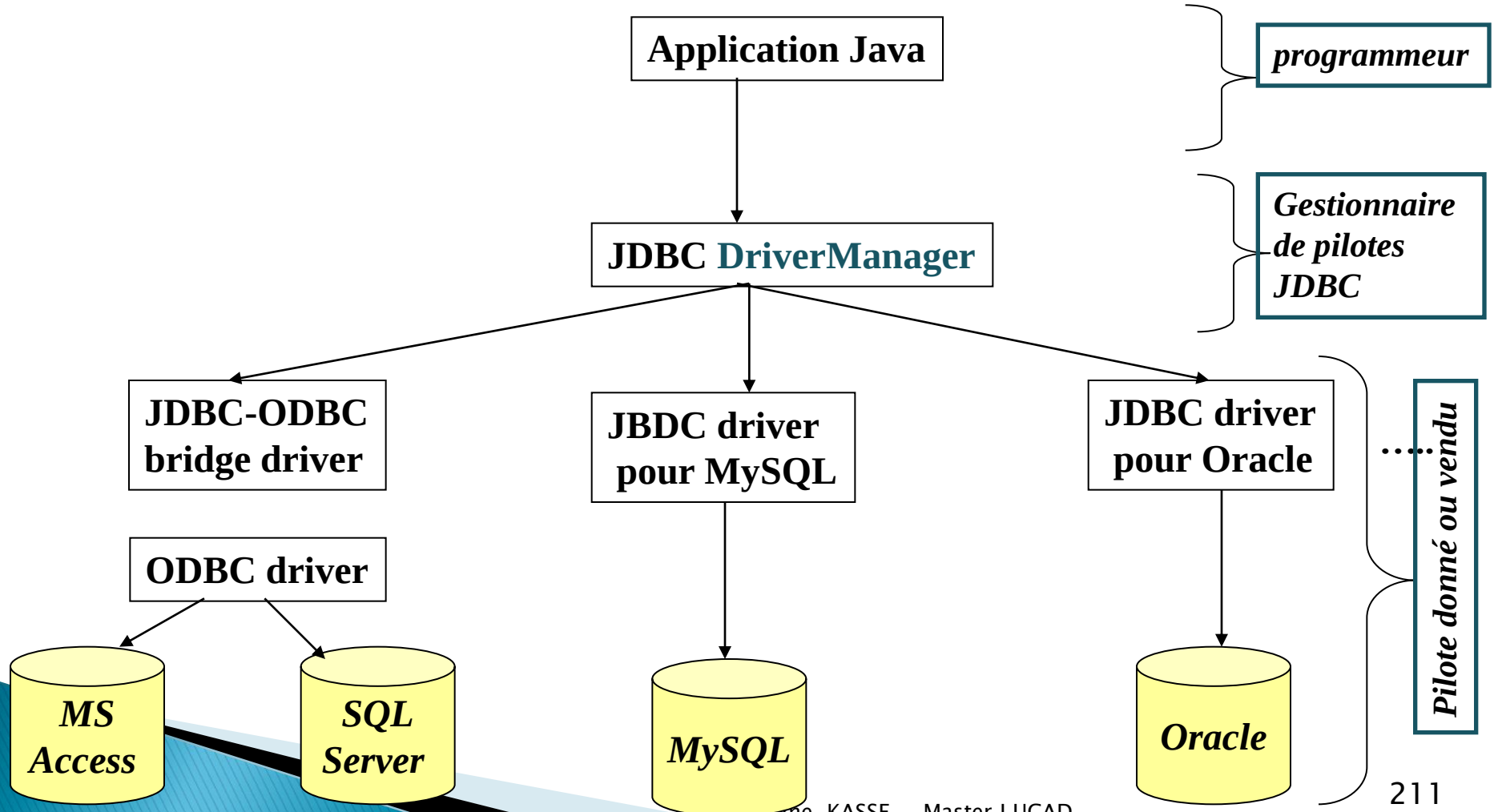
JDBC(Java DataBase Connectivity)

- JDBC permet d'accéder aux bases de données à l'aide
- du langage Java via des requêtes SQL (langage permettant de dialoguer avec un SGBDR).
- JDBC est fourni par le paquetage java.sql

- Dans une application JAVA utilisant une base de données avec l'interface JDBC, on trouvera généralement les étapes suivantes :

- 1. Connexion à la base de données
- 2. Émission de requêtes SQL vers la base
- 3. Réception et traitement des résultats de ces requêtes
- 4. Fermeture de la connexion

JDBC(Java DataBase Connectivity)



JDBC(Java DataBase Connectivity)

- Pour travailler avec un SGBD, il faut disposer de classes (driver) qui implémentent les interfaces de JDBC.
- JDBC est totalement indépendant de tout SGBD: la même application peut être utilisée pour accéder à une base Oracle, Sybase, MySQL,etc.

Chargement du pilote

- Un programme JDBC débute toujours par le chargement du pilote approprié pour la BD.
- Mais puisque le programme a la possibilité d'accéder à plusieurs types de BD, il peut avoir plusieurs pilotes.
- C'est au moment de la connexion à la BD que le DriverManager choisit alors le *bon* pilote.
- DriverManager = gestionnaire de tous les drivers chargés par un programme java

URLs : JDBC(Java DataBase Connectivity)

Après le chargement du driver, il faut localiser la BD en spécifiant son emplacement.
Pour chaque type de driver il existe un schéma de désignation de la base à atteindre avec JDBC:

Chaque URL est de la forme: jdbc: *sous-protocole :base_de_donnée*

Structure d'un programme: JDBC(Java DataBase Connectivity)

1. **Chargement du driver (= chargement de la classe du driver dans la JVM)**

Class.forName (String driverName);

```
Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver") ;
Class.forName ("oracle.jdbc.driver.OracleDriver") ;
Class.forName ("org.gjt.mm.mysql.Driver") ;
```

Quand une classe *Driver* est chargée, elle doit créer une instance d'elle même et s'enregistrer auprès du *DriverManager*.

Certains compilateurs refusent la notation précédente et demandent :

- Class.forName (" driver_name").newInstance ( ) ;
- Cette étape 1 constitue *l'enregistrement* du driver JDBC.

2. Établir une connexion à la base de données

Une fois le driver enregistré, une connexion peut être établie avec la BD.
pour obtenir une connexion à un SGBD, il faut faire une demande à la classe gestionnaire de drivers:

Demande permise par la méthode **getConnection (…)** de la classe **DriverManager**

Cette méthode peut prendre 3 arguments au plus:

- *l'URL vers la BD*
- *le nom de l'utilisateur de la base*
- *son mot de passe*

cette méthode renvoie un objet de type **Connection**.

```
Connection con =
  DriverManager.getConnection ("jdbc:odbc:MasterI", "login", "passwd");
```

3. Création de requêtes SQL en utilisant l'objet de type Connection (étape 2).

dans cette étape on crée en fait une zone de description de requêtes ie un espace où l'on pourra exécuter des opérations SQL.
cette zone est un objet de la classe **Statement** que l'on crée par la méthode **createStatement ( ) .**

```
Statement st = con.createStatement ( ) ;
```

Structure d'un programme: JDBC(Java DataBase Connectivity)

4. Envoi et exécution de requêtes.

Il existe trois types d'exécution de requêtes:

- **executeQuery (…):** pour les requêtes SELECT qui retournent un **ResultSet** (tuples),
- **executeUpdate (…):** pour les requêtes INSERT, UPDATE, DELETE, CREATE TABLE DROP TABLE qui retourne un entier **(int)** désignant le nombre de tuples traités.
- **execute ( ):** pour les procédures stockées (cas rares).

Les méthodes **executeQuery ( )** et **executeUpdate( )** de la classe **Statement** prennent comme argument une chaîne **(String)** indiquant la requête SQL à exécuter.

```
Statement st = con.createStatement ( );
ResultSet rs = st.executeQuery ("select * from Client") ;
```

Traitement des données retournées JDBC(Java DataBase Connectivity)

- si une requête ne peut s'exécuter ou qu'une erreur de syntaxe SQL a été
- détectée, l'exception **SQLException** est levée.
- Le driver JDBC effectue d'abord un premier accès à la BD pour découvrir les types des colonnes impliquées dans la requête puis un deuxième pour l'exécuter.

La méthode **next ( )** de **ResultSet** permet de parcourir itérativement ligne par ligne l'ensemble des tuples sélectionnés.

Cette méthode :

- retourne **false** si le dernier tuple est lu, **true** sinon,
- chaque appel fait avancer le curseur sur le tuple suivant,
- initialement, le curseur est positionné avant le premier tuple

Exécuter **next ( )** au moins une fois pour avoir le premier.

```
while (rs.next ( )) { //traitement tuple par tuple}
```

Traitement des données retournées JDBC(Java DataBase Connectivity)

Les *colonnes* d'une table de la BD sont référencées par **leur numéro** (commençant par 1) ou par leur **nom.**
L'accès aux valeurs des colonnes se fait par des méthodes de la forme **getXXX (…)** permettant la lecture de données du type **XXX** dans chaque colonne du tuple courant.

```
int val = rs.getInt (3) ; // acces  à la 3e colonne

String prod = rs.getString ("Produit") ;// acces à la colonne de nom Produit
```

Utilisation de JDBC pour MySQL

```
import java.sql .*;
public class TestJavaMySQL {
  static Connection con;
  static Statement st;
  static ResultSet rs;
  public static void main (String[] args) {

  try {  /*chargement du driver*/
          Class.forName("com.mysql.jdbc.Driver").newInstance ( )  ;
        }
  catch (Exception er_dv){System.out .println("Erreur driver:  "+e.getMessage() ) ;}
```

Utilisation de JDBC pour MySQL

```
 try {con = DriverManager.getConnection ("jdbc:mysql: //localhost
/MasterI","root","") ;}
  catch (Exception er_con ){System.out.println("Erreur de connexion "+
ez.getMessage ( ));}
  try {  st = con.createStatement() ; }
  catch (SQLException er_st){System.out.println ("Erreur de Statement
"+t.getMessage());}
  try {
        rs = st.executeQuery("select * from client") ;
           while (rs.next() )
             { System.out .println(rs.getObject (1)+" "+rs.getObject(2)+"
"+rs.getObject(3) ) ;}}
  catch (Exception er_rs)  {System.out .println("Erreur ResultSet "+er.getMessage ( )
); }
   try { rs.close ( ) ;        st.close ( );        con.close ( );}
   catch  (Exception d) { }
  }}
```

Correspondances SQL/JAVA

Si vous écrivez une instruction telle que String prenom=rs.getString(" prenom");
cela signifie que vous êtes convaincu que la colonne de nom PRENOM a été créée sous SQL avec le type *VARCHAR*. Autrement cette instruction causerais une erreur de runtime.
Donc il est bon de savoir la correspondance entre types SQL et types Java pour pouvoir manipuler convenablement les données lues.

Types SQL	Types Java	Méthodes
CHAR/ VARCHAR	String	getString ( )
INTEGER	int	getInt ( )
TINYINT	byte	getByte ( )
SMALLINT	short	getShort ( )

Correspondances SQL/JAVA

Types SQL	Types Java	Méthodes
BIGINT	long	getLong ( )
bit	boolean	getBoolean ( )
REAL	float	getFloat ( )
NUMERIC DECIMAL	java.math.BigDecimal	getBigDecimal ( )
DATE	java.sql.Date	getDate ( )
TIME	java.sql.Time	getTime ( )
TIMESTAMP	java.sql.TimeStamp	getTimeStamp ( )
FLOAT DOUBLE	double	getDouble ( )

Les MétaDonnées

On peut avoir besoin quelque fois des informations sur la structure et le schéma de la base de données elle-même. Ces informations sont appelées des Métadonnées.
Pour obtenir ces renseignements, il faut utiliser un objet de la classe DatabaseMetaData à partir de l'objet Connection. La méthode getMetaData ( ) de la classe Connection permet de créer cet objet.

```
DatabaseMetaData donneesBD = objet_con.getMetaData ( ) ;
```

A partir de cet objet, cette classe fournit beaucoup de renseignements sur la BD.
Par exemple:

```
DatabaseMetaData d = con.getMetaData ( ) ;
String nomSGBD       = d.getDatabaseProductName ( ) ;
String versionDriver  = d.getDriverVersion ( );
String nomDriver      = d.getDriverName ( );
```

Les MétaDonnées

On peut aussi avoir des informations sur les objets ResultSet générés:
- nombre de colonnes contenues dans le ResultSet,
- noms des colonnes
- types des colonnes,
- etc.

Pour récupérer ces Métadonnées, il faut créer un objet de la classe ResultSetMetaData grâce à la méthode getMetaData ( ) de la classe ResultSet.

On notera deux méthodes utiles dans la classe *ResultSetMetaData* :
1. int getColumnCount() qui donne le nombre de colonnes du *ResultSet*
2. String getColumnLabel(int i) qui donne le nom de la colonne i du *ResultSet* (i>=1)

```
ResultSetMetaData rsmd = objet_resultset.getMetaData ( );

int columnCount = rsmd.getColumnCount ( );
String columnLabel = rsmd.getColumnLabel ( i );
String columnType = rsmd.getColumnTypeName ( i ) ;
```

```
public void actionPerformed(ActionEvent av){
if(av.getSource()==b){try {    /*chargement du driver*/
            Class.forName("com.mysql.jdbc.Driver").newInstance ( )  ;

        }
 catch (Exception e){JOptionPane.showMessageDialog(null,"Erreur driver") ; }
 try { cn = DriverManager.getConnection ("jdbc:mysql://localhost/MasterI","root","") ; }
catch (Exception ez ){JOptionPane.showMessageDialog(null,"Erreur de Connexion") ;}
        try{st=cn.createStatement();}
  catch (SQLException t){JOptionPane.showMessageDialog(null,"Erreur d'operation");}
  try {
  int a=Integer.parseInt(jt1.getText());
  int e=Integer.parseInt(jt5.getText());
  //System.out.println(a+" "+e);
  String c=jt3.getText(),d=jt4.getText();
  System.out.println(c+" "+ d);
   rs=st.executeQuery("select * from client");
   while(rs.next())
u = st.executeUpdate("insert into client values('"+a+"','"+c+"','"+d+"','"+e+"')") ;
         System.out.println(u);
         if(u>0){JOptionPane.showMessageDialog(this,"Inscription effectuée avec succés")
;jt1.setText("");jt2.setText("");jt3.setText("");jt5.setText("");}
         else
         JOptionPane.showMessageDialog(this,"Vous n'êtes pas inscrits") ;
catch (Exception er)  { JOptionPane.showMessageDialog(this,"Erreur"); }
  try { rs.close ( ) ;        st.close ( );        cn.close ( );
        }
   catch (Exception d) { }   }}
```